

ORIX Education Series

ORIX 教育版四足机器狗 C++ SDK 开发手册

面向高性能嵌入式部署的专业级机器人开发平台

大算机器人

Brainstem SDK for C++

版本 v1.0.0

日期 2026 年 4 月

文档信息

项目	说明
文档名称	ORIX 教育版四足机器狗 C++ SDK 开发手册
版本号	v1.0.0
编写日期	2026 年 4 月
适用产品	ORIX 教育版四足机器狗
适用 SDK	orix (C++)

版本历史

版本	日期	作者	说明
v1.0.0	2026-04	大算机器人	正式发布

本文档为大算机器人内部技术文档，未经许可不得外传。

目录

前言	10
第一章 概述	11
1.1 产品定位	11
1.2 SDK 能做什么	11
1.3 典型开发流程	12
1.4 手册结构	12
第二章 快速开始	14
2.1 环境要求	14
2.2 安装与编译	14
2.2.1 方式一：vcpkg 安装依赖（推荐）	14
2.2.2 方式二：CMake 编译步骤	15
2.2.3 验证编译	15
2.3 三行代码控制机器人	15
2.4 首次连接检查清单	16
2.5 网络配置	16
2.6 验证连接程序	17
2.7 第一次完整操作	18
2.8 RAPI 与资源管理	19
2.9 常见问题	19
2.9.1 连接超时（DEADLINE_EXCEEDED）	20
2.9.2 walk() 没反应	20
2.9.3 编译失败：找不到 gRPC	20
2.9.4 操作被拒绝	20
第三章 理论基础	21
3.1 PD 位置控制	21
3.1.1 物理意义	21
3.1.2 稳定性条件	21
3.1.3 阻尼比分析	22

3.2	强化学习运动控制	22
3.2.1	策略网络	22
3.2.2	观测空间	22
3.2.3	动作空间与缩放	23
3.2.4	Sim-to-Real	23
3.3	姿态表示与坐标变换	23
3.3.1	四元数基础	23
3.3.2	投影重力计算	23
3.3.3	Hamilton 约定与 ROS 约定	24
3.4	有限状态机	24
3.4.1	形式化定义	24
3.4.2	安全不变量	24
3.5	控制频率与实时性	24
第四章	硬件概览	26
4.1	整体架构	26
4.2	关键参数	26
4.3	CAN 总线拓扑	27
4.4	传感器概览	27
第五章	坐标系与关节	28
5.1	机体坐标系	28
5.1.1	walk() 指令与坐标系的关系	28
5.2	关节编号总表	28
5.3	关节分组	29
5.3.1	按腿分组	29
5.3.2	按关节类型分组	29
5.4	典型姿态	29
5.4.1	站立姿态 (Standing Pose)	29
5.5	PD 控制参数	30
5.5.1	推理阶段 (行走时)	30
5.6	编程中的关节操作	30
5.6.1	读取关节数据	30
5.6.2	诊断单条腿	30
5.6.3	清除单腿故障	31
第六章	状态机与运动控制	32
6.1	状态机总览	32
6.1.1	五个状态	32

6.1.2	状态转换路径	32
6.2	状态转换详解	33
6.2.1	Grounded → Standing (站起)	33
6.2.2	Standing → Walking (开始行走)	33
6.2.3	Standing → Grounded (坐下)	33
6.3	非法操作处理	33
6.4	walk() 速度约定	34
6.5	控制权仲裁 (ControlArbiter)	34
6.6	典型使用流程	35
6.6.1	标准操作序列	35
6.6.2	实时数据监控	36
6.6.3	安全包装器	37
6.7	关键参数速查	38
第七章	实用开发指南	39
7.1	电池管理	39
7.2	数据记录与采集	40
7.2.1	记录 IMU 数据到 CSV	40
7.3	多线程流式订阅	41
7.4	集成 OpenCV 摄像头	42
7.4.1	视觉引导行走	43
7.5	自定义 Profile	44
7.6	CMake 工程集成最佳实践	45
第八章	传感器数据	47
8.1	传感器总览	47
8.2	IMU 数据	47
8.2.1	数据字段	47
8.2.2	订阅 IMU 数据	48
8.3	关节编码器	48
8.3.1	数据字段	48
8.3.2	订阅关节数据	49
8.4	电机状态	49
8.5	电压监控	49
8.6	综合示例：传感器数据记录到 CSV	50
第九章	API 参考	52
9.1	OrixClient 构造与连接	52
9.1.1	构造函数	52

9.1.2	析构函数	53
9.1.3	ping()	53
9.2	电机控制	53
9.2.1	enable()	54
9.2.2	disable()	54
9.3	运动指令	54
9.3.1	walk()	54
9.3.2	stand_up()	55
9.3.3	sit_down()	56
9.3.4	set_high_speed() / get_speed_mode()	56
9.4	状态查询	57
9.4.1	get_state()	57
9.4.2	get_voltage()	57
9.4.3	get_motor_status()	58
9.5	策略管理	59
9.5.1	get_profile()	59
9.5.2	switch_profile()	59
9.6	动作系统	60
9.6.1	list_gestures()	60
9.6.2	play_gesture()	60
9.7	诊断与标定	61
9.7.1	clear_motor_fault()	61
9.7.2	set_zero()	61
9.8	流式接口	62
9.8.1	listen_imu()	62
9.8.2	listen_joint()	63
9.8.3	listen_state()	63
9.9	OrixCamera API	64
9.9.1	构造方式	64
9.9.2	方法速查	64
9.10	数据类型参考	65
9.10.1	Vec3	65
9.10.2	Quat	65
9.10.3	ImuData	66
9.10.4	JointData	66
9.10.5	MotorInfo	66
9.10.6	ProfileInfo	67
9.10.7	GestureInfo	67

9.10.8 RobotState	67
9.11 异常类型	68
9.11.1 连接超时	68
9.11.2 常见错误码	68
9.11.3 Arbiter 控制权冲突	69
9.11.4 流式接口异常处理	69
第十章 安全指南	72
10.1 操作前检查清单	72
10.2 低压保护	73
10.3 关节限位保护	74
10.3.1 保护机制	74
10.3.2 触发后果	74
10.3.3 预防措施	74
10.4 电机故障处理	74
10.4.1 检测故障	74
10.4.2 清除故障	75
10.4.3 故障处理流程	75
10.4.4 反复出现的故障	75
10.5 急停程序	76
10.5.1 软件急停 (SDK)	76
10.5.2 遥控器急停	76
10.5.3 硬件断电	76
10.5.4 急停优先级	76
10.6 标准关机序列	76
10.7 Arbiter 控制仲裁安全	77
10.7.1 仲裁规则	77
10.7.2 对 SDK 开发者的影响	77
10.7.3 安全启示	77
10.8 禁止操作	78
10.9 故障诊断树	78
第十一章 常见问题	80
11.1 连接问题	80
11.1.1 Q: 连接超时 (TimeoutError)	80
11.1.2 Q: 连接被拒绝 (ConnectionError)	80
11.1.3 Q: IP 地址不确定	81
11.2 编译问题	81
11.2.1 Q: CMake 找不到 gRPC	81

11.2.2 Q: 链接错误: undefined reference to grpc	81
11.2.3 Q: C++17 标准不满足	82
11.3 电机不动	82
11.3.1 Q: enable() 后电机没反应	82
11.3.2 Q: 只有部分电机动, 其他不动	82
11.3.3 Q: enable() 后状态没变化	83
11.4 行走指令无效	83
11.4.1 Q: walk() 不生效	83
11.4.2 Q: 遥控器在线时 SDK 指令被忽略	83
11.5 机器人摔倒	83
11.5.1 Q: 行走中突然坐下	83
11.5.2 Q: 站起来后立即摔倒	84
11.6 策略切换失败	84
11.6.1 Q: switch_profile() 报错	84
11.6.2 Q: 策略名称不存在	85
11.6.3 Q: 切换策略后行为异常	85
11.7 IMU 数据异常	85
11.7.1 Q: 四元数值看起来不对	85
11.7.2 Q: 机器人静止但角速度不为零	86
11.7.3 Q: listen_imu() 频率不是 50Hz	86
11.8 流式接口问题	86
11.8.1 Q: listen_imu() 阻塞了主线程	86
11.8.2 Q: 流中断后如何重连	86
11.9 性能与延迟	87
11.9.1 Q: C++ SDK 比 Python 快多少?	87
11.9.2 Q: 50Hz 控制频率够用吗?	87
11.10 环境变量参考	87
11.11 摄像头问题	88
11.11.1 Q: OrixCamera 编译失败	88
11.11.2 Q: 远程访问摄像头	88
11.12 网络排障	88
11.13 提交 Bug 报告	89
11.13.1 必要信息	89
11.13.2 收集诊断信息	89
11.13.3 日志收集	90
11.13.4 联系方式	91
11.14 SSH 访问与系统服务	91
11.14.1 默认账户	91

11.14.2 brainstem 服务管理	91
11.14.3 无网络环境下的备选连接方式	92

前言

近年来，国家高度重视人工智能与机器人教育的发展。《新一代人工智能发展规划》明确要求在中小学和高等院校开设人工智能相关课程，《教育部关于加强和改进中小学实验教学的意见》强调动手实践和工程思维的培养，教育部《人工智能教育基地建设指南》进一步推动了 AI+ 机器人实验室在全国高校和中学的普及。在这一大背景下，实物机器人作为连接理论与实践的桥梁，正在成为教育体系的重要组成部分。

四足机器人是一类典型的高集成度教学平台。它融合了机械工程（腿部机构设计、运动学建模）、电子控制（伺服驱动、传感器融合）、嵌入式系统（实时控制、CAN 总线通信）、计算机视觉（姿态估计、环境感知）和人工智能（强化学习策略训练、sim-to-real 迁移）等多个学科的核心内容。相较于传统的机械臂或移动底盘，四足机器人的动力学更加丰富，控制问题更具挑战性，能够让学生直观感受到理论与工程之间的差距，也能作为强化学习、仿生运动控制、自主导航等前沿技术的实验载体。

ORIX 教育版四足机器狗是大算机器人面向教育场景推出的专业级开发平台。我们的设计遵循四个核心理念：**学以致用**——它不是展会上的观摩样品，而是一台可以编程、调参、训练自定义策略的真实机器人；**降低门槛**——几行 C++ 代码即可完成从连接到行走的全流程，学生不需要先掌握通信协议和控制理论；**不设天花板**——从入门的遥控行走到高级的 RL 策略训练、多传感器融合、SLAM 建图和自主导航，一个平台覆盖从本科到研究生的完整学习路径；**工业级可靠**——采用与工业机器人相同的 FSM 状态机、控制权仲裁和 E-Stop 安全机制，学生接触的是真实的工程实践。

本手册是 ORIX C++ SDK 的完整开发指南。相较于 Python SDK，C++ SDK 面向以下场景：**高性能嵌入式部署**——直接在机器人板端编译运行，消除网络延迟；**ROS2 节点开发**——作为 ROS2 包的组成部分，实现导航、SLAM 等高级功能；**低延迟控制**——无 GIL 限制，无解释器开销，适合对实时性要求极高的控制任务；**与机器视觉库直接集成**——与 OpenCV 等 C++ 库无缝协作，实现视觉引导行走。手册覆盖从环境安装到高级开发的完整流程，建议首次使用时按顺序阅读前三章（前言、概述、快速开始），完成首次连通后再根据兴趣阅读理论基础和 API 参考。

大算机器人技术团队 2026 年 4 月

Chapter 1

概述

本章导读

本章建立对 ORIX 平台的整体认识：它解决什么问题、能做什么、适合什么场景、后续章节如何组织。阅读后您将对整个 SDK 有全局认识，便于根据自己的目标决定详读哪些章节。

阅读建议：如果您想尽快跑通机器人控制，可跳到第 二 章；如果您关心控制理论，可直接读第 三 章。

1.1 产品定位

ORIX 教育版四足机器人是一个集成了运动控制、传感器融合、强化学习策略推理和安全机制的完整机器人平台。它由机器人本体（12 DOF 足式四足机器人）、C++ SDK (`orix`) 和开发文档三部分组成。机器人内置完整的控制系统，开发者通过 SDK 在开发电脑或机器人本体上编写 C++ 代码控制机器人，无需接触底层通信协议。

C++ SDK 特别适合需要极低延迟或嵌入式部署的场景。与 Python SDK 功能完全对等，但提供了原生性能、无 GIL 限制和与 OpenCV/ROS2 等 C++ 生态的直接集成能力。

1.2 SDK 能做什么

`orix` C++ SDK 提供了对机器人的全方位控制能力，主要包括以下五类接口：

- **运动控制** — 使能/禁用电机、站立、坐下、行走、切换速度模式、播放预设动作
- **传感器读取** — IMU 姿态数据流（50Hz）、关节编码器数据流、电机状态、母线电压
- **状态管理** — 查询 FSM 状态、订阅状态变化事件、切换运动策略
- **诊断与维护** — 读取电机故障码、清除故障、电机标零、健康检查
- **摄像头接口** — USB 摄像头拍照、录像、MJPEG 流媒体服务（需 OpenCV，可选模块）

整个 SDK 遵循简洁设计原则：客户代码完全不需要了解 gRPC、protobuf、状态机实现细节，所有复杂性隐藏在 `orix::OrixClient` 类后面。

```
1 #include "orix/orix.h"
2
3 int main() {
4     orix::OrixClient dog("192.168.66.190");
5     dog.enable();
6     dog.stand_up();
7     dog.walk(0.3);    // 前进 30% 速度
8     dog.sit_down();
9     dog.disable();
10    return 0;
11 }
```

1.3 典型开发流程

一个典型的 ORIX C++ 开发流程包含以下步骤：

1. 环境准备 — 开发机安装 C++17 编译器、CMake 3.16+、gRPC 1.50+ 和 Protobuf 3.21+（详见第 二 章）
2. 安装 SDK — 通过 vcpkg 安装依赖，或手动 cmake 编译
3. 验证连接 — 编译并运行验证程序，确认与机器狗通信正常
4. 编写控制代码 — 使用 `orix::OrixClient` 发送运动指令和读取传感器数据
5. 调试与迭代 — 根据需要切换策略、调整参数、查看日志
6. 高级开发 — 集成 OpenCV 计算机视觉、开发 ROS2 节点、训练自定义 RL 策略

1.4 手册结构

本手册共 11 章，按“入门 → 理论 → 参考 → 运维”的顺序组织：

章	主题	适合谁
1	概述（本章）	所有读者
2	快速开始	第一次使用的开发者
3	理论基础	希望理解控制原理的学生/研究人员
4	硬件概览	需要了解硬件参数的工程师
5	坐标系与关节	进行运动规划的开发者
6	状态机与运动控制	编写控制逻辑的开发者
7	实用开发指南	有实际项目需求的 C++ 开发者

章	主题	适合谁
8	传感器数据	进行数据采集和算法研究的开发者
9	API 参考	查阅接口细节时的所有开发者
10	安全指南	上手前必读
11	常见问题	遇到问题时查阅

首次使用建议按顺序阅读第 1-2 章（概述 + 快速开始），完成首次连通和基本控制验证。之后可根据需要跳读：如果关心控制理论，读第 3 章；如果要做运动规划，读第 5-6 章；如果要开发传感器融合或 RL 策略，读第 8 章；遇到具体接口问题查第 9 章。安全指南（第 10 章）在真机操作前必读。

Chapter 2

快速开始

本章导读

本章以”先跑通、再理解”为目标，带您完成编译、网络连通性验证和第一次完整动作闭环。即使您已经有机机器人经验，也建议至少执行一次连接检查程序。

阅读建议：按”环境要求 -> 安装与编译 -> 首次连接检查清单 -> 验证连接程序 -> 第一次完整操作”的顺序阅读。不要跳过安全提醒和状态机说明。

2.1 环境要求

在开始编译 SDK 之前，请确认您的开发环境满足以下最低要求。ORIX SDK 基于纯 gRPC 通信，不依赖 ROS 或其他中间件。

项目	要求
C++ 标准	C++17 或更高
编译器	GCC 9+ / Clang 10+ / MSVC 2019+
构建系统	CMake 3.16+
gRPC	>= 1.50.0
Protobuf	>= 3.21.0
操作系统	Linux / macOS / Windows
网络	开发机与机器人在同一局域网
端口	TCP 13145 (gRPC over HTTP/2)

注意：不需要安装 ROS、消息中间件或其他额外组件。SDK 通过纯 gRPC 直连机器人。

2.2 安装与编译

2.2.1 方式一：vcpkg 安装依赖（推荐）

使用 vcpkg 包管理器可以自动安装 gRPC 和 Protobuf 依赖：

```
1 # 安装 vcpkg (如果尚未安装)
2 git clone https://github.com/microsoft/vcpkg.git
```

```

3 cd vcpkg && ./bootstrap-vcpkg.sh
4
5 # 安装 gRPC 和 Protobuf
6 ./vcpkg install grpc protobuf
7
8 # 如果需要摄像头功能，同时安装 OpenCV
9 ./vcpkg install opencv4

```

2.2.2 方式二：CMake 编译步骤

```

1 cd sdk/cpp
2 mkdir build && cd build
3 cmake .. -DCMAKE_BUILD_TYPE=Release
4 cmake --build . --config Release

```

如果使用 vcpkg:

```

1 cmake .. -DCMAKE_BUILD_TYPE=Release \
2     -DCMAKE_TOOLCHAIN_FILE=/path/to/vcpkg/scripts/buildsystems/vcpkg
3     .cmake
4 cmake --build . --config Release

```

2.2.3 验证编译

编译成功后，运行示例程序验证：

```

1 # 编译后可执行文件在 build/examples/ 目录下
2 ./build/examples/01_hello_walk --help
3
4 # 或者运行 ping 测试
5 ./build/examples/01_hello_walk

```

2.3 三行代码控制机器人

下面这段代码适合作为“环境和链路是否通了”的最小验证，而不是正式的运动脚本。正式任务中，您仍然需要补齐使能、电机状态确认、站起和安全收尾等步骤。

```

1 #include "orix/orix.h"
2
3 int main() {
4     orix::OrixClient dog("192.168.66.190");

```

```

5     dog.walk(0.3); // 30% 速度前进
6     return 0;
7 }

```

这三行代码做了以下事情：

1. 包含 SDK 头文件
2. 通过 gRPC 连接到机器人（默认端口 13145）
3. 发送前进指令

注意：实际使用中，您需要先 `enable()` 使能电机并 `stand_up()` 站起来，才能执行 `walk()`。完整的操作流程见下方“第一次完整操作”。

2.4 首次连接检查清单

在第一次连接机器人之前，请逐项确认：

- ☐ 机器人已上电，电池电压正常（正常工作电压约 24V，低压保护 18V）
- ☐ 机器人主控板（S100P）已启动完成（约 30 秒）
- ☐ `brainstem` 服务已运行（`systemctl status brainstem`）
- ☐ 开发机与机器人在同一局域网（可以 `ping 192.168.66.190`）
- ☐ 遥控器已关闭或置于空闲状态（遥控器优先级高于 SDK）
- ☐ 机器人放置在平坦地面上，四足触地

2.5 网络配置

开发机（PC 或笔记本，IP 为 192.168.66.xxx）与 ORIX 机器人（IP 为 192.168.66.190）需处于同一局域网内，通过以太网或 WiFi 连接。SDK 使用 TCP 端口 13145 进行 gRPC 通信，无需额外中间件。

参数	默认值	说明
机器人 IP	192.168.66.190	S100P 主控板默认地址
gRPC 端口	13145	无需修改
协议	gRPC (HTTP/2)	无加密（局域网内使用）
超时	5.0 秒	可通过构造函数 <code>timeout</code> 参数调整

如果机器人 IP 不同，连接时指定即可：

```

1 orix::OrixClient dog("10.0.0.100", 13145, 5.0);

```


2.6 验证连接程序

首次连接机器狗时,建议使用以下验证程序逐项检查通信状态。将以下代码保存为 `verify_connection.cpp` 并编译运行:

```
1 #include "orix/orix.h"
2 #include <iostream>
3
4 int main() {
5     const std::string ROBOT_IP = "192.168.66.190";
6     std::cout << "正在连接 ORIX @ " << ROBOT_IP << ":13145 ...\\n";
7
8     try {
9         orix::OrixClient dog(ROBOT_IP, 13145, 3.0);
10
11         auto state = dog.get_state();
12         std::cout << "[OK] 状态机: " << orix::to_string(state) << "\\n";
13
14         auto voltages = dog.get_voltage();
15         if (!voltages.empty()) {
16             double avg = 0;
17             for (auto v : voltages) avg += v;
18             avg /= voltages.size();
19             std::cout << "[OK] 平均电压: " << avg << "V ("
20                 << voltages.size() << " 个电机)\\n";
21         }
22
23         auto motors = dog.get_motor_status();
24         int online = 0;
25         for (const auto& m : motors) if (m.online) ++online;
26         std::cout << "[OK] 电机在线: " << online << "/"
27             << motors.size() << "\\n";
28
29         auto profile = dog.get_profile();
30         std::cout << "[OK] 当前策略: " << profile.current << "\\n";
31
32         std::cout << "\\n--- 连接验证通过 ---\\n";
33
34     } catch (const orix::ConnectionError& e) {
35         std::cerr << "\\n[FAIL] 连接失败: " << e.what() << "\\n";
36         std::cerr << "排查步骤:\\n";
37         std::cerr << " 1. 确认机器人已上电\\n";
38         std::cerr << " 2. ping 192.168.66.190\\n";
```

```

39         std::cerr << "  3. 检查防火墙是否阻断 13145/TCP\n";
40         return 1;
41     }
42     return 0;
43 }

```

编译并运行：

```

1 g++ -std=c++17 -o verify_connection verify_connection.cpp \
2     -I/path/to/orix/include -L/path/to/orix/lib -lorix -lgrpc++ -
3     lprotobuf
4 ./verify_connection

```

预期输出：

```

正在连接 ORIX @ 192.168.66.190:13145 ...
[OK] 状态机: grounded
[OK] 平均电压: 25.5V (16 个电机)
[OK] 电机在线: 16/16
[OK] 当前策略: default

--- 连接验证通过 ---

```

2.7 第一次完整操作

确认连接验证通过后，尝试完整的站起-行走-坐下流程。这个流程比“三行代码”更接近真实使用场景，因为它包含了状态切换等待、停步和安全收尾，是后续编写演示程序和自动化测试的推荐模板。

```

1 #include "orix/orix.h"
2 #include <iostream>
3 #include <thread>
4 #include <chrono>
5
6 int main() {
7     orix::OrixClient dog("192.168.66.190");
8
9     dog.enable();
10    std::cout << "状态: " << orix::to_string(dog.get_state())
11              << "\n"; // grounded
12
13    dog.stand_up();
14    std::this_thread::sleep_for(std::chrono::seconds(4));

```

```

15     std::cout << "状态: " << orix::to_string(dog.get_state())
16           << "\n"; // standing
17
18     dog.walk(0.3);
19     std::this_thread::sleep_for(std::chrono::seconds(3));
20
21     dog.walk();          // 无参数 = 停止
22     std::this_thread::sleep_for(std::chrono::seconds(1));
23
24     dog.sit_down();
25     std::this_thread::sleep_for(std::chrono::seconds(4));
26     std::cout << "状态: " << orix::to_string(dog.get_state())
27           << "\n"; // grounded
28
29     dog.disable();
30     return 0; // dog 析构自动释放 gRPC channel
31 }

```

注意：首次操作建议在台架上进行，或有人在旁边准备随时用遥控器接管。遥控器具具有最高优先级，任何时候拨动遥控器都可以立即接管控制权。

2.8 RAII 与资源管理

Python SDK 使用 `with` 语句管理资源，C++ SDK 使用 RAII (Resource Acquisition Is Initialization) 模式。`OrixClient` 对象在析构时自动释放 gRPC 通道资源，无需手动调用 `close()`。

```

1 // Python 写法:
2 // with OrixClient("192.168.66.190") as dog:
3 //     state = dog.get_state()
4
5 // C++ 写法: OrixClient 是 RAII 对象，析构自动释放
6 {
7     orix::OrixClient dog("192.168.66.190");
8     auto state = dog.get_state();
9     std::cout << "状态: " << orix::to_string(state) << "\n";
10 } // dog 析构, gRPC channel 自动关闭

```

提示：在生产代码中，建议通过 `std::unique_ptr` 管理 `OrixClient` 的生命周期，或将其作为类成员，利用 RAII 确保资源安全释放。

2.9 常见问题

2.9.1 连接超时 (DEADLINE_EXCEEDED)

如果程序抛出 `orix::TimeoutError`，通常是网络配置或服务未启动导致的。

排查步骤：

1. 确认机器人已上电且启动完成
2. `ping 192.168.66.190` 确认网络连通
3. SSH 登录确认服务运行：`ssh sunrise@192.168.66.190 'systemctl status brainstem'`
4. 检查防火墙是否阻断了 13145/TCP

2.9.2 walk() 没反应

行走指令未生效是新手最常遇到的问题之一。必须按照状态机顺序操作：`enable()` → `stand_up()` → 等待站稳 → `walk()`。

```
1 auto state = dog.get_state();
2 std::cout << orix::to_string(state) << "\n";
3 // 如果不是 "standing", walk() 不会生效
```

2.9.3 编译失败：找不到 gRPC

CMake 找不到 gRPC 时，需要手动指定安装路径：

```
1 cmake .. -DCMAKE_PREFIX_PATH=/path/to/grpc/install
2 # 或使用 vcpkg toolchain
3 cmake .. -DCMAKE_TOOLCHAIN_FILE=/path/to/vcpkg/scripts/buildsystems/vcpkg
   .cmake
```

2.9.4 操作被拒绝

当状态机正在执行过渡动画时，新的运动指令会被拒绝。C++ SDK 会抛出 `orix::InvalidStateError` 异常。

```
1 try {
2     dog.walk(0.3);
3 } catch (const orix::InvalidStateError& e) {
4     std::cerr << "状态错误：" << e.what() << "\n";
5     // 状态机正在过渡中，请等待过渡完成
6 }
```

Chapter 3

理论基础

本章导读

本章介绍 ORIX 机器狗控制系统涉及的核心理论，包括 PD 控制、强化学习策略、姿态表示和状态机形式化定义。理解这些理论有助于更好地使用 SDK 进行二次开发和算法研究。

阅读建议：如果您只需要快速上手控制机器狗，可以跳过本章，直接阅读 API 参考章节。

3.1 PD 位置控制

四足机器人的每个关节电机均采用 PD（比例-微分）位置控制器。PD 控制器根据目标位置与当前位置的偏差，以及目标速度与当前速度的偏差，计算驱动关节所需的力矩。

其基本控制律为：

$$\tau = K_p (q_{\text{target}} - q_{\text{current}}) + K_d (\dot{q}_{\text{target}} - \dot{q}_{\text{current}}) \quad (3.1)$$

其中：

- τ —— 关节输出力矩（N·m）
- K_p —— 比例增益（位置刚度），单位 N·m/rad
- K_d —— 微分增益（速度阻尼），单位 N·m·s/rad
- $q_{\text{target}}, q_{\text{current}}$ —— 目标与当前关节角度（rad）
- $\dot{q}_{\text{target}}, \dot{q}_{\text{current}}$ —— 目标与当前关节角速度（rad/s）

3.1.1 物理意义

K_p 决定了关节的**刚度**： K_p 越大，关节对位置偏差的响应越强，如同弹簧越硬。 K_d 决定了关节的**阻尼**： K_d 越大，关节运动越平滑，振荡越少，如同液压阻尼器。

3.1.2 稳定性条件

PD 控制器稳定的必要条件为：

$$K_p > 0, \quad K_d > 0 \quad (3.2)$$

3.1.3 阻尼比分析

将单关节建模为二阶系统 $I\ddot{q} + K_d\dot{q} + K_pq = 0$ (I 为关节转动惯量), 可定义阻尼比 ζ 和固有频率 ω_n :

$$\omega_n = \sqrt{\frac{K_p}{I}} \quad (3.3)$$

$$\zeta = \frac{K_d}{2\sqrt{I \cdot K_p}} \quad (3.4)$$

不同阻尼比对应不同的动态行为:

- $\zeta < 1$ (欠阻尼): 关节到达目标位置时会出现振荡, 适用于需要快速响应的场景, 但可能引起机械抖动。
- $\zeta = 1$ (临界阻尼): 关节以最快速度到达目标位置且不超调。这是理论上最优的阻尼比。
- $\zeta > 1$ (过阻尼): 关节缓慢趋近目标位置, 无超调但响应速度慢。

在实际机器人中, 通常选取略欠阻尼 ($\zeta \approx 0.7 \sim 0.9$) 的参数, 以兼顾响应速度和稳定性。

3.2 强化学习运动控制

ORIX 采用基于强化学习 (Reinforcement Learning, RL) 的方法训练四足步态策略。策略网络在高保真物理仿真器 (Isaac Lab) 中通过数百万次交互学习, 然后直接部署到真实机器人上 (Sim-to-Real)。

3.2.1 策略网络

策略网络 π 是一个将观测映射到动作的神经网络:

$$\mathbf{a}_t = \pi(\mathbf{o}_t) \quad (3.5)$$

其中 $\mathbf{o}_t$ 为 t 时刻的观测向量, $\mathbf{a}_t$ 为输出动作。

3.2.2 观测空间

单帧观测向量维度为 57, 概念上由以下部分组成:

$$\mathbf{o} = \begin{bmatrix} \omega \\ \mathbf{g}_{\text{proj}} \\ \mathbf{c}_{\text{cmd}} \\ \mathbf{q}_{\text{pos}} \\ \dot{\mathbf{q}} \\ \mathbf{a}_{\text{prev}} \\ \mathbf{b}_{\text{contact}} \end{bmatrix} \in \mathbb{R}^{57} \quad (3.6)$$

各分量含义如下：

分量	维度	说明
ω	3	陀螺仪角速度（机体坐标系）
$\mathbf{g}_{\text{proj}}$	3	投影重力向量（见 3.3 节）
$\mathbf{c}_{\text{cmd}}$	3	速度指令 (v_x, v_y, ω_z)
$\mathbf{q}_{\text{pos}}$	12	关节位置（12 腿关节，rad）
$\dot{\mathbf{q}}$	12	关节速度（rad/s）
$\mathbf{a}_{\text{prev}}$	12	上一时刻动作输出
$\mathbf{b}_{\text{contact}}$	12	足端接触布尔量

3.2.3 动作空间与缩放

策略网络输出 12 个腿关节的位置增量。原始网络输出 $\mathbf{a} \in [-1, 1]^{12}$ 经缩放后变为目标关节角度：

$$q_{\text{target},i} = a_i \times \text{actionScale}_i + q_{\text{stand},i} \quad (3.7)$$

3.2.4 Sim-to-Real

策略在 Isaac Lab 仿真环境中训练，使用域随机化（Domain Randomization）缩小仿真与现实的差距。训练完成后，策略网络导出为 ONNX 格式，部署到真实机器人的嵌入式处理器上进行实时推理。

3.3 姿态表示与坐标变换

机器人的姿态（朝向）在三维空间中通常用四元数表示，它避免了欧拉角的万向锁问题。

3.3.1 四元数基础

单位四元数 $\mathbf{q}$ 定义为：

$$\mathbf{q} = (w, x, y, z), \quad \|\mathbf{q}\| = \sqrt{w^2 + x^2 + y^2 + z^2} = 1 \quad (3.8)$$

3.3.2 投影重力计算

投影重力（Projected Gravity）是将世界坐标系的重力方向变换到机体坐标系，用于让策略感知机器人当前的倾斜状态：

$$\mathbf{g}_{\text{body}} = R^{\top} \begin{pmatrix} 0 \\ 0 \\ -1 \end{pmatrix} \quad (3.9)$$

3.3.3 Hamilton 约定与 ROS 约定

两种常见的四元数排列约定：

- **Hamilton 约定** (ORIX / Protobuf): (w, x, y, z) —— 实部 w 在前
- **ROS 约定** (ROS2 / tf2): (x, y, z, w) —— 实部 w 在后

在 ORIX 系统内部统一使用 Hamilton 约定。当与 ROS 系统交互时，需手动调整字段顺序。

3.4 有限状态机

ORIX 的运动控制流程由一个有限状态机 (Finite State Machine, FSM) 管理。FSM 确保机器人在任何时刻都处于明确定义的状态，且所有状态转换都经过安全校验。

3.4.1 形式化定义

FSM 形式化定义为五元组：

$$\text{FSM} = (S, \Sigma, \delta, s_0, F) \quad (3.10)$$

$$S = \{\text{Zero, Grounded, Standing, Walking, Transitioning}\} \quad (3.11)$$

$$\Sigma = \{\text{init, standUp, sitDown, walk, stop, fault}\} \quad (3.12)$$

$$s_0 = \text{Zero} \quad (3.13)$$

3.4.2 安全不变量

FSM 设计中最重要安全不变量：

从任意状态出发，收到 `fault` 信号后，系统必须经由安全坐下动画回到 `Grounded` 状态。

3.5 控制频率与实时性

ORIX 的主控制循环以 50 Hz 固定频率运行，即每个控制周期为：

$$T = \frac{1}{50 \text{ Hz}} = 20 \text{ ms} \quad (3.14)$$

C++ SDK 相较于 Python SDK 的一大优势在于：C++ 没有 GIL（全局解释器锁）限制，可以在多线程中真正并行执行计算密集型任务。这使得 C++ SDK 特别适合以下场景：

- 在独立线程中以 50Hz 订阅 IMU 数据，同时在线程中执行运动控制
- 将传感器数据处理和运动指令发送完全解耦
- 在机器人本体上直接运行，消除网络延迟

Chapter 4

硬件概览

本章导读

本章介绍 ORIX 教育版四足机器狗的硬件组成，包括主控板、电机、传感器和通信接口。了解硬件参数有助于正确配置 SDK 和排查硬件相关问题。

阅读建议：首次使用只需了解整体架构和关键参数（电压、端口、关节数量），详细参数可在需要时查阅。

4.1 整体架构

ORIX 教育版四足机器狗由以下核心组件组成：

组件	型号/规格	说明
主控板	地瓜机器人 S100P (RDK X5)	AI 算力 128 TOPS (Nash BPU)
IMU	HI91 惯性测量单元	串口 /dev/ttyUSB1, 约 50Hz
电机	Robstride 伺服电机 × 16	CAN 总线通信
电池	7 串锂电池	正常 24V, 低压保护 18V
通信	RJ45 以太网 + WiFi	gRPC 端口 13145
遥控器	YUNZHUO 无线遥控器	最高控制优先级

4.2 关键参数

参数	值
自由度	12 DOF (每腿 3 旋转关节)
控制频率	50 Hz (20ms 周期)
推理引擎	ONNX Runtime
通信协议	gRPC over HTTP/2
默认 IP	192.168.66.190
gRPC 端口	13145
SSH 端口	22 (用户 sunrise / 密码 sunrise)
正常电压	约 24V

参数	值
低压保护	18V

4.3 CAN 总线拓扑

16 个 Robstride 伺服电机通过 4 路 CAN 总线与主控板通信，每路 CAN 总线连接一条腿的 4 个电机。CAN 通信由 brainstem 控制栈在底层管理，SDK 用户无需直接操作 CAN 总线。

4.4 传感器概览

传感器	位置	接口	数据
IMU (HI91)	机身中心	串口	角速度 + 四元数
关节编码器	每个电机内置	CAN	位置/速度/力矩
电压传感器	每个电机内置	CAN	母线电压
温度传感器	每个电机内置	CAN	电机温度

Chapter 5

坐标系与关节

本章导读

本章用于统一方向、编号和姿态语言，避免多人协作时出现”左右相反””关节号对不上””旋转方向理解不一致”这类低级但高成本的问题。

阅读建议：开始写控制代码前，至少应熟悉机体坐标系、四条腿的编号分组以及 `walk()` 的三个速度轴向定义。

5.1 机体坐标系

ORIX 使用右手坐标系，原点位于机身几何中心：

轴	正方向	说明
X	前方 (Forward)	机器人头部方向
Y	左方 (Left)	机器人左侧方向
Z	上方 (Up)	垂直向上

5.1.1 `walk()` 指令与坐标系的关系

参数	正值	负值	对应轴
<code>vx</code>	前进	后退	+X / -X
<code>vy</code>	向左	向右	+Y / -Y
<code>vyaw</code>	逆时针	顺时针	绕 +Z 轴

5.2 关节编号总表

ORIX 全部 16 个关节的编号和 URDF 名称：

索引	名称	腿	关节类型	URDF 名称
0	FR Hip	前右	髋 (横滚)	<code>fr_hip_joint</code>
1	FR Thigh	前右	大腿 (俯仰)	<code>fr_thigh_joint</code>
2	FR Calf	前右	小腿 (俯仰)	<code>fr_calf_joint</code>

索引	名称	腿	关节类型	URDF 名称
3	FL Hip	前左	髋 (横滚)	fl_hip_joint
4	FL Thigh	前左	大腿 (俯仰)	fl_thigh_joint
5	FL Calf	前左	小腿 (俯仰)	fl_calf_joint
6	RR Hip	后右	髋 (横滚)	rr_hip_joint
7	RR Thigh	后右	大腿 (俯仰)	rr_thigh_joint
8	RR Calf	后右	小腿 (俯仰)	rr_calf_joint
9	RL Hip	后左	髋 (横滚)	rl_hip_joint
10	RL Thigh	后左	大腿 (俯仰)	rl_thigh_joint
11	RL Calf	后左	小腿 (俯仰)	rl_calf_joint

5.3 关节分组

5.3.1 按腿分组

腿	关节索引
前右 (FR)	[0, 1, 2]
前左 (FL)	[3, 4, 5]
后右 (RR)	[6, 7, 8]
后左 (RL)	[9, 10, 11]

5.3.2 按关节类型分组

关节类型	索引	功能
所有 Hip	[0, 3, 6, 9]	髋关节, 控制腿的内外摆动
所有 Thigh	[1, 4, 7, 10]	大腿关节, 控制腿的前后摆动
所有 Calf	[2, 5, 8, 11]	小腿关节, 控制膝盖弯曲

5.4 典型姿态

5.4.1 站立姿态 (Standing Pose)

站立姿态定义了机器人正常直立时所有关节的目标角度:

```

1 // 站立姿态 (12 个关节)
2 double standing_pose[12] = {
3     0.0, -0.65, 1.5,    // FR: hip, thigh, calf

```

```

4      0.0,  0.65, -1.5,    // FL: hip, thigh, calf
5      0.0, -0.65,  1.5,    // RR: hip, thigh, calf
6      0.0,  0.65, -1.5,    // RL: hip, thigh, calf
7  };

```

注意：注意左右对称：FR 和 RR 的符号相同，FL 和 RL 的符号相同（镜像关系）。

5.5 PD 控制参数

5.5.1 推理阶段（行走时）

索引	关节	Kp	Kd	actionScale
0, 3, 6, 9	Hip	80	10	0.15
1, 4, 7, 10	Thigh	100	10	0.25
2, 5, 8, 11	Calf	120	15	0.25

5.6 编程中的关节操作

5.6.1 读取关节数据

```

1 // 实时流式读取关节数据
2 dog.listen_joint([](const orix::JointData& j) -> bool {
3     std::cout << "关节 " << j.id
4                 << ": pos=" << j.position << "rad"
5                 << " vel=" << j.velocity << "rad/s"
6                 << " trq=" << j.torque << "Nm\n";
7     return true; // 继续接收
8 });

```

5.6.2 诊断单条腿

```

1 // 检查前右腿 (FR) 的 4 个电机
2 auto motors = dog.get_motor_status();
3 for (const auto& m : motors) {
4     if (m.id >= 0 && m.id <= 3) { // FR 腿
5         std::cout << "  关节" << m.id
6                 << ": online=" << m.online
7                 << " temp=" << m.temperature << "C\n";
8     }
9 }

```

```
8     }  
9 }
```

5.6.3 清除单腿故障

```
1 // 只清除前右腿的电机故障  
2 dog.clear_motor_fault({0, 1, 2, 3});
```

Chapter 6

状态机与运动控制

本章导读

本章解释”什么状态下什么指令才会生效”，是理解运动控制、排查动作不响应以及设计安全流程的核心章节。

阅读建议：如果您刚开始接触 SDK，建议先读”状态机总览””状态转换详解”和”典型使用流程”，再回头查各类异常和边界条件。

6.1 状态机总览

ORIX 的运动控制由一个严格的有限状态机管理。所有 SDK 指令必须遵循状态机规则，非法的状态转换会被拒绝并抛出 `orix::InvalidStateError` 异常。

6.1.1 五个状态

状态	SDK 返回值	说明
Zero	<code>RobotState::Zero</code>	上电初始态，FSM 尚未初始化
Grounded	<code>RobotState::Grounded</code>	坐姿/趴地，安全状态
Standing	<code>RobotState::Standing</code>	直立站稳，可接收行走指令
Walking	<code>RobotState::Walking</code>	行走中，RL 策略以 50Hz 推理
Transitioning	<code>RobotState::Transitioning</code>	过渡中，拒绝新的运动指令

6.1.2 状态转换路径

起始状态	触发条件	中间状态	目标状态
Zero	系统初始化	–	Grounded
Grounded	<code>stand_up()</code>	Transitioning	Standing
Standing	<code>walk(vx, vy, vyaw)</code>	–	Walking
Walking	<code>walk()</code> 停步	Transitioning	Standing
Standing	<code>sit_down()</code>	Transitioning	Grounded
Walking	<code>sit_down()</code>	先 StandUp 再 SitDown	Grounded

起始状态	触发条件	中间状态	目标状态
任意状态	Fault 故障	自动过渡	Grounded

6.2 状态转换详解

6.2.1 Grounded → Standing（站起）

项目	说明
触发	<code>dog.stand_up()</code>
耗时	$150 \text{ ticks} \times 20\text{ms} = \mathbf{3 \text{ 秒}}$
过程	从当前姿态线性插值到 <code>standingPose</code>

6.2.2 Standing → Walking（开始行走）

项目	说明
触发	<code>dog.walk(vx, vy, vyaw)</code>
耗时	即时切换
过程	RL 策略接管，以 50Hz 推理输出关节指令

6.2.3 Standing → Grounded（坐下）

项目	说明
触发	<code>dog.sit_down()</code>
耗时	$150 \text{ ticks} \times 20\text{ms} = \mathbf{3 \text{ 秒}}$
过程	从 <code>standingPose</code> 线性插值到 <code>sittingPose</code> (全零)

6.3 非法操作处理

当前状态	发送的指令	结果
Grounded	<code>walk()</code>	抛出 <code>InvalidStateError</code>
Grounded	<code>sit_down()</code>	忽略（已在坐姿）
Standing	<code>stand_up()</code>	忽略（已站立）
Transitioning	任何运动指令	抛出 <code>InvalidStateError</code>

当前状态	发送的指令	结果
Zero	任何运动指令	忽略

最佳实践：在发送指令前先查询状态。

```

1 auto state = dog.get_state();
2 if (state == orix::RobotState::Standing) {
3     dog.walk(0.3);
4 } else if (state == orix::RobotState::Grounded) {
5     dog.stand_up();
6 }

```

6.4 walk() 速度约定

```

1 dog.walk(vx, vy, vyaw);

```

参数	范围	正值	负值	零值
vx	[-1.0, 1.0]	前进 (+X)	后退 (-X)	不动
vy	[-1.0, 1.0]	向左 (+Y)	向右 (-Y)	不动
vyaw	[-1.0, 1.0]	逆时针 (+Z)	顺时针 (-Z)	不转

```

1 dog.walk(0.5);           // 前进
2 dog.walk(-0.3);          // 后退
3 dog.walk(0.0, 0.3);      // 向左平移
4 dog.walk(0.0, 0.0, 0.3); // 原地逆时针转
5 dog.walk(0.3, 0.0, 0.2); // 前进 + 左转
6 dog.walk();              // 停步

```

6.5 控制权仲裁 (ControlArbiter)

ORIX 支持多个控制源同时存在，通过 ControlArbiter 仲裁控制权。遥控器始终具有最高优先级。

场景	行为
SDK 控制中，遥控器摇杆动	SDK 被立即抢占，遥控器接管
遥控器控制中，SDK 发指令	SDK 指令被忽略
遥控器松手不动	3 秒超时后自动释放，SDK 恢复控制权

场景	行为
SDK 控制中，遥控器关机	SDK 保持控制权，不受影响

6.6 典型使用流程

6.6.1 标准操作序列

完整的机器人操作序列，包含诊断检查、运动和安全收尾：

```

1  #include "orix/orix.h"
2  #include <algorithm>
3  #include <chrono>
4  #include <iostream>
5  #include <thread>
6
7  int main() {
8      orix::OrixClient dog("192.168.66.190");
9
10     // 1. 诊断检查
11     auto state = dog.get_state();
12     std::cout << "初始状态: " << orix::to_string(state) << "\n"; //
        grounded
13
14     auto voltages = dog.get_voltage();
15     double avg_v = 0;
16     for (auto v : voltages) avg_v += v;
17     avg_v /= voltages.size();
18     std::cout << "电池电压: " << avg_v << "V\n";
19
20     auto motors = dog.get_motor_status();
21     int offline = 0;
22     for (const auto& m : motors) if (!m.online) ++offline;
23     std::cout << "电机在线: " << (16 - offline) << "/16\n";
24
25     // 2. 使能电机
26     dog.enable();
27
28     // 3. 站起 (3 秒过渡)
29     dog.stand_up();
30     std::this_thread::sleep_for(std::chrono::seconds(4));
31     state = dog.get_state();
32     std::cout << "状态: " << orix::to_string(state) << "\n"; // standing

```

```

33
34 // 4. 行走
35 dog.walk(0.3); // 前进
36 std::this_thread::sleep_for(std::chrono::seconds(3));
37 dog.walk(0.3, 0.0, 0.2); // 前进 +
    左转
38 std::this_thread::sleep_for(std::chrono::seconds(2));
39 dog.walk(); // 停步
40
41 // 5. 坐下 (3 秒过渡)
42 dog.sit_down();
43 std::this_thread::sleep_for(std::chrono::seconds(4));
44 state = dog.get_state();
45 std::cout << "状态: " << orix::to_string(state) << "\n"; // grounded
46
47 // 6. 禁用电机
48 dog.disable();
49 return 0;
50 }

```

6.6.2 实时数据监控

同时监控 IMU 和状态变化，主线程执行运动控制：

```

1 #include "orix/orix.h"
2 #include <atomic>
3 #include <chrono>
4 #include <iostream>
5 #include <thread>
6
7 int main() {
8     orix::OrixClient dog("192.168.66.190");
9     std::atomic<bool> running{true};
10
11     // 后台线程监听 IMU
12     std::thread imu_thread([&] {
13         dog.listen_imu([&](const orix::ImuData& d) -> bool {
14             std::cout << "\rIMU: gx=" << d.gyroscope.x
15                 << " gy=" << d.gyroscope.y
16                 << " gz=" << d.gyroscope.z
17                 << std::flush;
18             return running.load();
19         });

```

```

20     });
21
22     // 后台线程监听状态变化
23     std::thread state_thread([&] {
24         dog.listen_state([&](orix::RobotState s) -> bool {
25             std::cout << "\n[STATE] -> " << orix::to_string(s) << "\n";
26             return running.load();
27         });
28     });
29
30     // 主线程执行运动
31     dog.enable();
32     dog.stand_up();
33     std::this_thread::sleep_for(std::chrono::seconds(4));
34     dog.walk(0.3);
35     std::this_thread::sleep_for(std::chrono::seconds(5));
36     dog.sit_down();
37     std::this_thread::sleep_for(std::chrono::seconds(4));
38     dog.disable();
39
40     running = false;
41     imu_thread.join();
42     state_thread.join();
43     return 0;
44 }

```

6.6.3 安全包装器

在生产代码中使用 RAII + 异常处理确保安全退出：

```

1  #include "orix/orix.h"
2  #include <chrono>
3  #include <thread>
4
5  void safe_run(orix::OrixClient& dog) {
6      dog.enable();
7      dog.stand_up();
8      std::this_thread::sleep_for(std::chrono::seconds(4));
9
10     // ... 业务逻辑 ...
11     dog.walk(0.3);
12     std::this_thread::sleep_for(std::chrono::seconds(5));
13 }

```

```

14
15 int main() {
16     orix::OrixClient dog("192.168.66.190");
17     try {
18         safe_run(dog);
19     } catch (const std::exception& e) {
20         // 任何异常都执行安全关机
21     }
22     // 安全收尾（无论是否异常）
23     try { dog.walk(); } catch (...) {}
24     try { dog.sit_down(); } catch (...) {}
25     // 等待进入 Grounded...
26     for (int i = 0; i < 50; ++i) {
27         if (dog.get_state() == orix::RobotState::Grounded) break;
28         std::this_thread::sleep_for(std::chrono::milliseconds(200));
29     }
30     try { dog.disable(); } catch (...) {}
31     return 0;
32 }

```

6.7 关键参数速查

参数	值	说明
控制频率	50 Hz (20ms)	主控制循环和推理频率
gRPC 端口	13145	SDK 连接端口
站起耗时	3 秒 (150 ticks)	Grounded → Standing
坐下耗时	3 秒 (150 ticks)	Standing → Grounded
仲裁超时	3 秒	遥控器释放控制权等待时间
低压阈值	18V	低于此值触发保护坐下
启动超时	10 秒	FSM 等待 Grounded 超时
关机超时	8 秒	安全关机等待超时
RPC 超时	5 秒	SDK 默认 RPC 超时

Chapter 7

实用开发指南

本章导读

本章汇集了在实际 C++ 开发中经常遇到的场景和最佳实践。包括电池管理、数据记录、多线程流式订阅、集成 OpenCV 视觉、自定义 Profile 和 CMake 工程集成等实用模式。每个场景都给出可直接使用的代码模板。

阅读建议：本章不涉及新的 API，而是展示如何组合已有 API 解决真实问题。初次阅读可以略读，实际开发时作为速查参考。

7.1 电池管理

机器狗采用 7 串锂电池，正常工作电压约 24V，低压保护阈值 18V。在长时间运行的应用中，建议实现主动电池监控。

```
1 #include "orix/orix.h"
2 #include <iostream>
3 #include <algorithm>
4
5 double estimate_battery_percent(orix::OrixClient& dog) {
6     auto voltages = dog.get_voltage();
7     if (voltages.empty()) return -1.0;
8
9     // 过滤有效读数
10    double sum = 0;
11    int count = 0;
12    for (auto v : voltages) {
13        if (v > 0) { sum += v; ++count; }
14    }
15    if (count == 0) return -1.0;
16
17    double avg_v = sum / count;
18    // 线性插值: 29.4V=100%, 18V=0%
19    double pct = std::clamp((avg_v - 18.0) / (29.4 - 18.0) * 100.0,
20                            0.0, 100.0);
21    return pct;
```

```
22 }
23
24 int main() {
25     orix::OrixClient dog("192.168.66.190");
26     double pct = estimate_battery_percent(dog);
27     auto voltages = dog.get_voltage();
28     double min_v = *std::min_element(voltages.begin(), voltages.end());
29
30     std::cout << "电量: " << pct << "% ("
31               << min_v << "V)\n";
32
33     if (pct < 30.0) {
34         std::cout << "低电量, 建议收工充电.\n";
35     }
36     return 0;
37 }
```

7.2 数据记录与采集

7.2.1 记录 IMU 数据到 CSV

```
1 #include "orix/orix.h"
2 #include <chrono>
3 #include <fstream>
4 #include <iostream>
5
6 int main() {
7     orix::OrixClient dog("192.168.66.190");
8     std::ofstream csv("imu_log.csv");
9     csv << "timestamp_ms,gx,gy,gz,qw,qx,qy,qz\n";
10
11     int count = 0;
12     auto t0 = std::chrono::steady_clock::now();
13
14     dog.listen_imu([&](const orix::ImuData& d) -> bool {
15         csv << d.timestamp_ms << ","
16             << d.gyroscope.x << "," << d.gyroscope.y
17             << "," << d.gyroscope.z << ","
18             << d.quaternion.w << "," << d.quaternion.x
19             << "," << d.quaternion.y << ","
20             << d.quaternion.z << "\n";
21         ++count;
22     });
23 }
```



```

22     auto elapsed = std::chrono::steady_clock::now() - t0;
23     return elapsed < std::chrono::seconds(10);
24 });
25
26 std::cout << "记录完成: " << count
27           << " 帧 -> imu_log.csv\n";
28 return 0;
29 }

```

7.3 多线程流式订阅

C++ SDK 的流式接口（listen_imu、listen_joint、listen_state）是阻塞调用，建议在独立线程中运行。C++ 没有 GIL 限制，多线程可以真正并行执行。

```

1  #include "orix/orix.h"
2  #include <atomic>
3  #include <chrono>
4  #include <iostream>
5  #include <thread>
6
7  int main() {
8      orix::OrixClient dog("192.168.66.190");
9      std::atomic<bool> running{true};
10
11     // 在独立线程中订阅 IMU
12     std::thread imu_thread([&] {
13         dog.listen_imu([&](const orix::ImuData& d) -> bool {
14             std::cout << "gyro_x=" << d.gyroscope.x
15                       << " qw=" << d.quaternion.w
16                       << " t=" << d.timestamp_ms << "ms\n";
17             return running.load();
18         });
19     });
20
21     // 在独立线程中订阅状态变化
22     std::thread state_thread([&] {
23         dog.listen_state([&](orix::RobotState s) -> bool {
24             std::cout << "[STATE] -> "
25                       << orix::to_string(s) << "\n";
26             return running.load();
27         });
28     });

```

```
29
30     // 主线程执行运动控制
31     dog.enable();
32     dog.stand_up();
33     std::this_thread::sleep_for(std::chrono::seconds(4));
34
35     dog.walk(0.3);
36     std::this_thread::sleep_for(std::chrono::seconds(5));
37
38     dog.sit_down();
39     std::this_thread::sleep_for(std::chrono::seconds(4));
40     dog.disable();
41
42     // 停止所有流
43     running = false;
44     imu_thread.join();
45     state_thread.join();
46
47     return 0;
48 }
```

提示：使用 `std::atomic<bool>` 控制流的生命周期，是 C++ 多线程流式订阅的标准模式。callback 返回 `false` 时流会自动停止。

7.4 集成 OpenCV 摄像头

OrixCamera 模块提供了与 OpenCV 集成的摄像头接口。以下示例展示基本的拍照和录像功能：

```
1  #include "orix/orix_camera.h"
2  #include <iostream>
3
4  int main() {
5      // USB 摄像头 (设备索引 0, 640x480, 30fps)
6      orix::OrixCamera cam(0, 640, 480, 30);
7      cam.open();
8
9      // 拍照
10     cam.save_photo("photo.jpg");
11     std::cout << "照片已保存: photo.jpg\n";
12
13     // 录制 10 秒视频
```

```
14     cam.start_recording("output.mp4", 10.0);
15     std::cout << "正在录制 10 秒...\n";
16     // 录制在后台线程进行, 等待完成
17     std::this_thread::sleep_for(std::chrono::seconds(11));
18
19     // 启动 MJPEG 流服务
20     cam.stream_mjpeg(8080);
21     std::cout << "MJPEG 流: http://localhost:8080/stream\n";
22     std::cout << "快照: http://localhost:8080/snapshot\n";
23
24     // 保持运行, 按需停止
25     std::this_thread::sleep_for(std::chrono::seconds(30));
26
27     cam.stop_stream();
28     cam.close();
29     return 0;
30 }
```

7.4.1 视觉引导行走

结合 OpenCV 实现基于颜色追踪的行走控制:

```
1  #include "orix/orix.h"
2  #include "orix/orix_camera.h"
3  #include <opencv2/opencv.hpp>
4  #include <iostream>
5
6  int main() {
7      orix::OrixClient dog("192.168.66.190");
8      orix::OrixCamera cam(0, 640, 480, 30);
9      cam.open();
10
11      dog.enable();
12      dog.stand_up();
13      std::this_thread::sleep_for(std::chrono::seconds(4));
14
15      try {
16          for (int i = 0; i < 500; ++i) {
17              cv::Mat frame = cam.read();
18              if (frame.empty()) continue;
19
20              // 检测红色区域
21              cv::Mat hsv;
```

```

22         cv::cvtColor(frame, hsv, cv::COLOR_BGR2HSV);
23         cv::Mat mask;
24         cv::inRange(hsv, cv::Scalar(0, 100, 100),
25                     cv::Scalar(10, 255, 255), mask);
26
27         auto moments = cv::moments(mask);
28         if (moments.m00 > 500) {
29             int cx = static_cast<int>((
30                 moments.m10 / moments.m00);
31             int center = frame.cols / 2;
32             double turn = -(cx - center)
33                         / static_cast<double>(center) * 0.3;
34             dog.walk(0.2, 0.0, turn);
35         } else {
36             dog.walk(); // 停止
37         }
38
39         std::this_thread::sleep_for(
40             std::chrono::milliseconds(30));
41     }
42 } catch (...) {}
43
44 dog.walk();
45 dog.sit_down();
46 std::this_thread::sleep_for(std::chrono::seconds(4));
47 dog.disable();
48 cam.close();
49 return 0;
50 }

```

7.5 自定义 Profile

Profile 是机器狗的运动参数集合。自定义 Profile 的 JSON 文件结构:

```

{
    "name": "my_custom",
    "description": "自定义策略",
    "modelPath": "model/my_policy.onnx",
    "standingPose": [0.0, -0.65, 1.5, 0.0, ...],
    "sittingPose": [0.0, 0.0, 0.0, 0.0, ...],
    "inferKp": [...], "inferKd": [...],
    "actionScale": [0.125, 0.25, 0.25, 5.0]
}

```

```
}

```

部署和切换:

```
1 # 上传到机器狗
2 scp my_custom.json sunrise@192.168.66.190:~/profiles/

1 // 查看可用策略
2 auto profile = dog.get_profile();
3 std::cout << "当前: " << profile.current << "\n";
4 for (const auto& name : profile.available) {
5     std::cout << " 可用: " << name << "\n";
6 }
7
8 // 切换策略 (必须在 Grounded 状态)
9 auto new_profile = dog.switch_profile("my_custom");
10 std::cout << "已切换到: " << new_profile.current << "\n";

```

7.6 CMake 工程集成最佳实践

在您自己的 CMake 项目中集成 ORIX SDK 的推荐方式:

```
1 cmake_minimum_required(VERSION 3.16)
2 project(my_robot_app LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 17)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6
7 # 查找 ORIX SDK
8 find_package(orix REQUIRED)
9
10 # 查找 gRPC 和 Protobuf
11 find_package(gRPC REQUIRED)
12 find_package(Protobuf REQUIRED)
13
14 # 可选: OpenCV (如果使用 OrixCamera)
15 find_package(OpenCV 4.5 QUIET)
16
17 add_executable(my_app main.cpp)
18 target_link_libraries(my_app PRIVATE
19     orix::orix
20     gRPC::grpc++)

```

```
21     protobuf::libprotobuf
22 )
23
24 # 如果使用 OrixCamera
25 if(OpenCV_FOUND)
26     target_link_libraries(my_app PRIVATE
27         orix::orix_camera
28         ${OpenCV_LIBS}
29     )
30 endif()
```

提示：使用 `vcpkg` 作为包管理器时，在 `cmake` 命令中添加 `-DCMAKE_TOOLCHAIN_FILE=.../vcpkg.cmake` 即可自动解析所有依赖路径。

Chapter 8

传感器数据

本章导读

本章重点说明各类传感器数据的含义、频率和 C++ 代码读取方式，适合做数据记录、状态监测、教学实验和上层算法开发时查阅。

阅读建议：如果您只想快速拿到稳定可用的数据流，建议优先阅读”IMU 数据””关节编码器”和”综合示例”。

8.1 传感器总览

传感器	硬件	数据内容	接口方法	频率	传输方式
IMU	HI91	角速度 + 四元数	<code>listen_imu()</code>	~50 Hz	串口 → gRPC 流
关节	Robstride	位置/速度/力矩	<code>listen_joint()</code>	~50 Hz	CAN → gRPC 流
电机	Robstride	在线/温度/故障	<code>get_motor_status()</code>	按需	CAN → gRPC
电压	Robstride	16 路母线电压	<code>get_voltage()</code>	按需	CAN → gRPC
状态	软件	FSM 状态	<code>listen_state()</code>	事件	gRPC 流

8.2 IMU 数据

8.2.1 数据字段

字段	类型	单位	说明
<code>gyroscope.x</code>	double	rad/s	绕机体 X 轴角速度 (Roll)
<code>gyroscope.y</code>	double	rad/s	绕机体 Y 轴角速度 (Pitch)
<code>gyroscope.z</code>	double	rad/s	绕机体 Z 轴角速度 (Yaw)
<code>quaternion.w</code>	double	无量纲	四元数实部
<code>quaternion.x</code>	double	无量纲	四元数虚部 i
<code>quaternion.y</code>	double	无量纲	四元数虚部 j
<code>quaternion.z</code>	double	无量纲	四元数虚部 k
<code>timestamp_ms</code>	double	ms	数据时间戳

8.2.2 订阅 IMU 数据

```

1  #include "orix/orix.h"
2  #include <iostream>
3  #include <iomanip>
4
5  int main() {
6      orix::OrixClient dog("192.168.66.190");
7
8      std::cout << std::fixed << std::setprecision(4);
9      int count = 0;
10
11     dog.listen_imu([&](const orix::ImuData& d) -> bool {
12         ++count;
13         if (count % 25 == 0) { // 每 0.5 秒打印一次
14             std::cout << "gyro: ["
15                 << d.gyroscope.x << ", "
16                 << d.gyroscope.y << ", "
17                 << d.gyroscope.z << "]" << "
18                 << "quat: w=" << d.quaternion.w
19                 << " t=" << d.timestamp_ms << "ms\n";
20         }
21         return count < 500; // 约 10 秒
22     });
23
24     std::cout << "共接收 " << count << " 帧\n";
25     return 0;
26 }

```

注意：ORIX SDK 使用 Hamilton 约定 (w, x, y, z)，ROS 使用 (x, y, z, w)。如需与 ROS 对接，请手动调整字段顺序。

8.3 关节编码器

8.3.1 数据字段

字段	类型	单位	说明
id	int	—	关节编号 (0–15)
position	double	rad	当前角位置
velocity	double	rad/s	当前角速度

字段	类型	单位	说明
torque	double	Nm	当前力矩
status	int	–	状态码 (0 = 正常)

8.3.2 订阅关节数据

```

1 int count = 0;
2 dog.listen_joint([&](const orix::JointData& j) -> bool {
3     std::cout << "Joint " << j.id
4         << ": pos=" << j.position
5         << " vel=" << j.velocity
6         << " trq=" << j.torque << "\n";
7     return ++count < 500;
8 });

```

8.4 电机状态

```

1 auto motors = dog.get_motor_status();
2 for (const auto& m : motors) {
3     std::string st = m.online ? "在线" : "离线";
4     std::cout << "Joint " << m.id << ": " << st
5         << " temp=" << m.temperature << "C"
6         << " V=" << m.voltage
7         << " errors=" << m.errors.size() << "\n";
8 }

```

8.5 电压监控

```

1 auto voltages = dog.get_voltage();
2 double min_v = *std::min_element(voltages.begin(),
3                                   voltages.end());
4 double max_v = *std::max_element(voltages.begin(),
5                                   voltages.end());
6 std::cout << "电压范围: " << min_v << "V ~ "
7     << max_v << "V\n";
8
9 if (min_v < 18.0) {
10     std::cerr << "WARNING: 低压保护即将触发!\n";

```

```
11 }
```

8.6 综合示例：传感器数据记录到 CSV

以下示例同时在两个线程中订阅 IMU 和关节数据，分别写入 CSV 文件：

```
1  #include "orix/orix.h"
2  #include <atomic>
3  #include <chrono>
4  #include <fstream>
5  #include <iostream>
6  #include <thread>
7
8  int main() {
9      orix::OrixClient dog("192.168.66.190");
10     std::atomic<bool> running{true};
11
12     // IMU 记录线程
13     std::thread imu_thread([&] {
14         std::ofstream csv("imu_log.csv");
15         csv << "timestamp_ms,gx,gy,gz,qw,qx,qy,qz\n";
16         dog.listen_imu([&](const orix::ImuData& d) -> bool {
17             csv << d.timestamp_ms << ","
18                 << d.gyroscope.x << "," << d.gyroscope.y
19                 << "," << d.gyroscope.z << ","
20                 << d.quaternion.w << "," << d.quaternion.x
21                 << "," << d.quaternion.y << ","
22                 << d.quaternion.z << "\n";
23             return running.load();
24         });
25         std::cout << "IMU 记录完成 -> imu_log.csv\n";
26     });
27
28     // 关节记录线程
29     std::thread joint_thread([&] {
30         std::ofstream csv("joint_log.csv");
31         csv << "joint_id,position,velocity,torque,status\n";
32         dog.listen_joint([&](const orix::JointData& j) -> bool {
33             csv << j.id << "," << j.position << ","
34                 << j.velocity << "," << j.torque << ","
35                 << j.status << "\n";
36             return running.load();
```

```
37     });
38     std::cout << "关节记录完成 -> joint_log.csv\n";
39     });
40
41     // 主线程等待 30 秒
42     std::cout << "开始记录 30 秒...\n";
43     std::this_thread::sleep_for(std::chrono::seconds(30));
44
45     running = false;
46     imu_thread.join();
47     joint_thread.join();
48     std::cout << "记录完成.\n";
49
50     return 0;
51 }
```

Chapter 9

API 参考

本章提供 ORIX C++ SDK (orix) 全部公开接口的完整参考。

本章导读

本章按”查手册”的方式组织，不强调教学顺序，而强调接口签名、前置条件、返回值和常见调用场景。

阅读建议：第一次阅读时建议按”构造与连接 → 电机控制 → 运动指令 → 状态查询 → 诊断与标定 → 流式接口”的顺序浏览，形成稳定的接口心智模型。

9.1 OrixClient 构造与连接

9.1.1 构造函数

```
explicit OrixClient(  
    std::string host    = "192.168.66.190",  
    int          port    = 13145,  
    double       timeout = 5.0);
```

创建 OrixClient 实例并建立 gRPC channel (惰性连接，首次 RPC 时才真正握手)。构造成功只说明网络通道已建立，不代表机器人已具备执行动作的安全条件；真正发送运动指令前，仍应检查状态、电压和控制权。

参数	类型	默认值	说明
host	std::string	"192.168.66.190"	机器人 IP 地址
port	int	13145	gRPC 服务端口 (无需修改)
timeout	double	5.0	默认 RPC 超时秒数

说明：OrixClient 不可拷贝，可移动。推荐将其作为类成员或通过 std::unique_ptr 管理生命周期，在一个任务中尽量复用同一连接，避免反复创建和销毁。

```
1 // 默认连接  
2 orix::OrixClient dog("192.168.66.190");  
3  
4 // 自定义参数
```

```

5 orix::OrixClient dog("10.0.0.100", 13145, 10.0);
6
7 // unique_ptr 管理
8 auto dog = std::make_unique<orix::OrixClient>("192.168.66.190");

```

9.1.2 析构函数

```
~OrixClient();
```

析构时自动释放 gRPC channel 和所有相关资源 (RAII)。无需手动调用 `close()`。Python SDK 的 `with` 语句等价于 C++ 的 RAII 作用域:

```

1 // Python 写法: with OrixClient(...) as dog: ...
2 // C++ 写法:
3 {
4     orix::OrixClient dog("192.168.66.190");
5     auto state = dog.get_state();
6 } // dog 析构, channel 自动关闭

```

9.1.3 ping()

```
bool ping();
```

检测机器人是否可达。超时 2 秒, 不抛异常。

项目	说明
返回值	<code>true</code> = 连接正常; <code>false</code> = 不可达或超时
超时	固定 2 秒, 不受构造时 <code>timeout</code> 参数影响
异常	不抛出任何异常

```

1 if (!dog.ping()) {
2     std::cerr << "无法连接到机器人, 请检查网络和服务状态.\n";
3     return 1;
4 }
5 std::cout << "连接正常.\n";

```

9.2 电机控制

9.2.1 enable()

```
void enable();
```

使能全部 16 个电机。调用后电机通电并进入闭环位置控制。

项目	说明
前置条件	gRPC 连接正常，控制权未被遥控器占用
异常	ConnectionError ——超时或连接失败

```
1 dog.enable();
2 // 此时状态仍为 Grounded，电机通电但保持坐姿
3 // 需要调用 stand_up() 才能站起
```

9.2.2 disable()

```
void disable();
```

禁用全部电机（安全停止）。电机立即断电，关节自由落体。

项目	说明
前置条件	无（任何状态下均可调用）
建议	在 <i>Grounded</i> 状态下调用，避免机器人从站立姿态跌落
异常	ConnectionError ——超时或连接失败

```
1 // 安全关机序列
2 dog.walk();           // 停步
3 dog.sit_down();       // 坐下
4 // 等待 Grounded ...
5 dog.disable();        // 禁用电机
```

9.3 运动指令

9.3.1 walk()

```
void walk(double vx = 0.0, double vy = 0.0, double vyaw = 0.0);
```

发送行走速度指令。

参数	类型	默认值	范围	说明
vx	double	0.0	[-1, 1]	前后速度。正值 = 前进，负值 = 后退
vy	double	0.0	[-1, 1]	左右速度。正值 = 向左，负值 = 向右
vyaw	double	0.0	[-1, 1]	旋转速度。正值 = 逆时针，负值 = 顺时针

项目	说明
前置条件	机器人处于 Standing 或 Walking 状态
停步	调用 <code>dog.walk()</code> 无参数，或发送 (0, 0, 0)
异常	<code>ConnectionError</code> 、 <code>TimeoutError</code> 、 <code>InvalidStateError</code>

速度参数为归一化值，不是物理速度。`vx=1.0` 表示当前策略允许的最大前进速度，实际物理速度取决于所加载的运动策略。

```

1 // 前进
2 dog.walk(0.5);
3
4 // 向左平移
5 dog.walk(0.0, 0.3);
6
7 // 原地逆时针旋转
8 dog.walk(0.0, 0.0, 0.3);
9
10 // 前进 + 左转
11 dog.walk(0.3, 0.0, 0.2);
12
13 // 原地停步
14 dog.walk();

```

9.3.2 stand_up()

```
void stand_up();
```

从坐姿过渡到站立姿态。

项目	说明
前置条件	机器人处于 Grounded 状态，电机已使能
状态变化	Grounded → Transitioning → Standing
耗时	约 3 秒（150 个控制周期 × 20ms）

项目	说明
异常	ConnectionError、InvalidStateError

```

1 dog.enable();
2 dog.stand_up();
3 std::this_thread::sleep_for(std::chrono::seconds(4)); // 等待站稳
4 auto state = dog.get_state();
5 // state == RobotState::Standing

```

9.3.3 sit_down()

```
void sit_down();
```

从站立过渡到坐姿。如果正在行走，会先停步再坐下。

项目	说明
前置条件	机器人处于 Standing 或 Walking 状态
状态变化	Standing/Walking → Transitioning → Grounded
耗时	约 3 秒；从 Walking 坐下约 6 秒（先站稳再坐）
异常	ConnectionError、InvalidStateError

```

1 dog.sit_down();
2 std::this_thread::sleep_for(std::chrono::seconds(4));
3 auto state = dog.get_state();
4 // state == RobotState::Grounded

```

9.3.4 set_high_speed() / get_speed_mode()

```

void          set_high_speed(bool enabled = true);
std::string   get_speed_mode();

```

开启/关闭高速模式（最大 2.5 m/s）。仅在开阔平坦场地使用。

项目	说明
前置条件	机器人处于 Standing 状态
返回值	get_speed_mode() 返回 "normal" 或 "high_speed"
异常	ConnectionError、InvalidStateError


```

1 dog.stand_up();
2 std::this_thread::sleep_for(std::chrono::seconds(4));
3
4 dog.set_high_speed(true);           // 启用高速模式
5 dog.walk(1.0);                      // 全速前进 (最大 2.5 m/s)
6
7 auto mode = dog.get_speed_mode(); // "high_speed"
8
9 dog.set_high_speed(false);          // 恢复正常速度

```

9.4 状态查询

9.4.1 get_state()

```
RobotState get_state();
```

查询当前机器人 FSM 状态。

返回值	说明
RobotState::Zero	初始化/未使能
RobotState::Grounded	坐姿（安全状态）
RobotState::Standing	站立，可接收 walk() 或 sit_down()
RobotState::Walking	行走中，RL 策略以 50Hz 推理驱动步态
RobotState::Transitioning	过渡中（站起/坐下），拒绝新的运动指令

```

1 auto state = dog.get_state();
2 std::cout << "当前状态: " << orix::to_string(state) << "\n";
3
4 if (state == orix::RobotState::Standing) {
5     dog.walk(0.3);
6 } else if (state == orix::RobotState::Grounded) {
7     dog.stand_up();
8 }

```

9.4.2 get_voltage()

```
std::vector<double> get_voltage();
```

读取全部 16 个电机的母线电压（单位 V）。

项目	说明
返回值	长度 16 的 double 向量，按关节编号 0-15 排列
索引	对应关节编号 0-15（同 Matrix4 关节顺序）
低压阈值	18V，低于此值系统自动触发保护坐下

```

1 auto voltages = dog.get_voltage();
2 double min_v = *std::min_element(voltages.begin(), voltages.end());
3 double max_v = *std::max_element(voltages.begin(), voltages.end());
4 std::cout << "电压范围: " << min_v << "V ~ " << max_v << "V\n";
5
6 for (int i = 0; i < (int)voltages.size(); ++i) {
7     std::string warn = voltages[i] < 18.0 ? " [低压!]" : "";
8     std::cout << " Joint " << i << ": " << voltages[i] << "V" << warn <<
9         "\n";
10 }

```

9.4.3 get_motor_status()

```
std::vector<MotorInfo> get_motor_status();
```

获取全部 16 个电机的健康状态。

```

1 auto motors = dog.get_motor_status();
2 int online = 0, faults = 0;
3 for (const auto& m : motors) {
4     if (m.online) ++online;
5     if (!m.errors.empty()) ++faults;
6     if (!m.online || !m.errors.empty()) {
7         std::cout << "Joint " << m.id
8             << ": online=" << m.online
9             << " errors=" << m.errors.size()
10            << " temp=" << m.temperature << "C\n";
11    }
12 }
13 std::cout << "在线: " << online << "/16 故障: " << faults << "/16\n";

```

故障码解读：

场景	表现	处理方式
online=false	电机未响应 CAN 通信	检查 CAN 线缆和电源
errors 非空	电机报告故障	调用 clear_motor_fault() 清除
temperature > 80	电机过热	降低负载或暂停运动，等待冷却
voltage < 42	母线电压过低	立即充电，系统将触发低压保护

9.5 策略管理

9.5.1 get_profile()

```
ProfileInfo get_profile();
```

获取当前运动策略信息及可用策略列表。

```
1 auto profile = dog.get_profile();
2 std::cout << "当前策略: " << profile.current << "\n";
3 std::cout << "可用策略: ";
4 for (size_t i = 0; i < profile.available.size(); ++i) {
5     std::cout << profile.available[i];
6     if (i < profile.descriptions.size())
7         std::cout << "(" << profile.descriptions[i] << ")";
8     std::cout << " ";
9 }
10 std::cout << "\n";
```

9.5.2 switch_profile()

```
ProfileInfo switch_profile(const std::string& name);
```

切换运动策略。

项目	说明
参数 name	目标策略名称，必须在 get_profile().available 列表中
前置条件	机器人处于 Grounded 状态
返回值	切换后的 ProfileInfo (current 为新策略名)
异常	ConnectionError、InvalidStateError——策略不存在或状态不对

```
1 // 先查看可用策略
2 auto profile = dog.get_profile();
```

```

3 for (const auto& p : profile.available)
4     std::cout << "    可用: " << p << "\n";
5
6 // 必须先坐下
7 dog.sit_down();
8 std::this_thread::sleep_for(std::chrono::seconds(4));
9
10 // 切换策略
11 auto new_profile = dog.switch_profile("mini");
12 std::cout << "已切换到: " << new_profile.current << "\n";

```

9.6 动作系统

9.6.1 list_gestures()

```
std::vector<GestureInfo> list_gestures();
```

列出所有可用预设动作。

```

1 auto gestures = dog.list_gestures();
2 for (const auto& g : gestures) {
3     std::cout << "    " << g.name
4               << ": " << g.description
5               << " (" << g.duration_ms << "ms)\n";
6 }

```

9.6.2 play_gesture()

```
void play_gesture(const std::string& name, double timeout_s = 30.0);
```

播放预设动作。执行期间机器人处于 Transitioning 状态，不接受新运动指令。

项目	说明
前置条件	机器人处于 Standing 状态
参数 name	动作名称（见 list_gestures()）
参数 timeout_s	最长等待时间（秒），默认 30s
异常	TimeoutError、InvalidStateError

```

1 dog.stand_up();
2 std::this_thread::sleep_for(std::chrono::seconds(4));

```

```

3
4 // 播放鞠躬动作（预计 2000ms，多等 5 秒余量）
5 dog.play_gesture("bow", 7.0);
6
7 // 播放跳舞动作
8 dog.play_gesture("dance", 35.0);

```

9.7 诊断与标定

9.7.1 clear_motor_fault()

```
void clear_motor_fault(const std::vector<int>& joint_ids = {});
```

清除电机故障码。

项目	说明
参数 joint_ids	要清除的关节编号列表（0-15）；空向量 = 清除全部
前置条件	无

```

1 // 清除所有电机故障
2 dog.clear_motor_fault();
3
4 // 只清除前右腿的 3 个关节
5 dog.clear_motor_fault({0, 1, 2});
6
7 // 只清除单个关节
8 dog.clear_motor_fault({5});

```

9.7.2 set_zero()

```
void set_zero();
```

将当前关节位置标定为零点并保存。

项目	说明
前置条件	机器人处于 Grounded 状态
用途	出厂标定或维修后重新校准零位
注意	错误的零位会导致运动异常，非必要不调用

```

1 // 确认在 Grounded 状态下执行
2 auto state = dog.get_state();
3 if (state == orix::RobotState::Grounded) {
4     dog.set_zero();
5     std::cout << "零位标定完成。当前位置已设为零点。\\n";
6 } else {
7     std::cout << "当前状态: " << orix::to_string(state)
8               << ", 标零需要在 Grounded 状态。\\n";
9 }

```

9.8 流式接口

所有 `listen_*` 方法接受一个回调函数 (`std::function<bool(...)>`)，回调返回 `true` 继续接收，返回 `false` 停止流。这些方法是阻塞调用，应在独立的 `std::thread` 中运行。

9.8.1 `listen_imu()`

```
void listen_imu(const std::function<bool(const ImuData&)>& callback);
```

订阅实时 IMU 数据流（50Hz）。

项目	说明
产出	<code>ImuData</code> （含 <code>gyroscope</code> 、 <code>quaternion</code> 、 <code>timestamp_ms</code> ）
频率	~50 Hz
阻塞	是（在独立线程调用；callback 返回 <code>false</code> 时退出）

```

1 std::atomic<bool> running{true};
2
3 std::thread imu_thread([&] {
4     dog.listen_imu([&](const orix::ImuData& d) -> bool {
5         std::cout << "gyro: (" << d.gyroscope.x
6                   << ", " << d.gyroscope.y
7                   << ", " << d.gyroscope.z << ")\\n";
8         return running.load();
9     });
10 });
11
12 std::this_thread::sleep_for(std::chrono::seconds(5));
13 running = false;
14 imu_thread.join();

```

9.8.2 listen_joint()

```
void listen_joint(const std::function<bool(const JointData&)>& callback);
```

订阅实时关节数据流（逐关节上报，50Hz）。

项目	说明
产出	JointData（含 id、position、velocity、torque、status）
频率	~50 Hz（逐关节上报，每帧一个关节）
阻塞	是

```
1 // 要获取完整的 16 关节快照，需自行缓存最近一帧
2 std::array<orix::JointData, 16> latest{};
3
4 dog.listen_joint([&](const orix::JointData& j) -> bool {
5     if (j.id >= 0 && j.id < 16)
6         latest[j.id] = j;
7     return true;
8 });
```

9.8.3 listen_state()

```
void listen_state(const std::function<bool(RobotState)>& callback);
```

订阅 FSM 状态变更事件（仅在状态切换时触发）。

项目	说明
产出	RobotState 枚举值
频率	事件驱动（仅状态变化时触发，不定频）
阻塞	是

```
1 std::atomic<bool> running{true};
2
3 std::thread state_thread([&] {
4     dog.listen_state([&](orix::RobotState s) -> bool {
5         std::cout << "[STATE] -> " << orix::to_string(s) << "\n";
6         return running.load();
7     });
8 });
```

```
7     });  
8 };
```

9.9 OrixCamera API

OrixCamera 是可选的摄像头模块（依赖 OpenCV ≥ 4.5 ），需要链接 orix_camera 库。

```
// 包含头文件  
#include "orix/orix_camera.h"
```

9.9.1 构造方式

```
// 本地 USB 摄像头  
explicit OrixCamera(int source=0, int width=640, int height=480, int fps  
                    =30);  
  
// 远程 RTSP / MJPEG URL  
explicit OrixCamera(const std::string& url);
```

```
1 // USB 摄像头（设备索引 0）  
2 orix::OrixCamera cam(0, 640, 480, 30);  
3  
4 // 远程 RTSP 流  
5 orix::OrixCamera cam("rtsp://192.168.66.190:8554/cam");
```

9.9.2 方法速查

方法	返回值	说明
open()	bool	打开摄像头；失败抛 CameraOpenError
close()	void	释放摄像头及所有资源
is_opened()	bool	摄像头是否已打开
read()	cv::Mat	读取一帧（BGR），失败返回空 Mat
get_frame_size()	pair<int,int>	(width, height)
save_photo(path)	bool	拍照并保存
start_recording(path, duration, codec)	void	启动后台录像
stop_recording()	void	停止录像
is_recording()	bool	是否正在录制

方法	返回值	说明
<code>stream_mjpeg(port, host)</code>	<code>void</code>	启动 MJPEG HTTP 流服务器
<code>stop_stream()</code>	<code>void</code>	停止流服务器
<code>is_streaming()</code>	<code>bool</code>	流服务器是否运行

```

1 #include "orix/orix_camera.h"
2
3 // 拍照
4 {
5     orix::OrixCamera cam(0, 640, 480, 30);
6     cam.open();
7     cam.save_photo("photo.jpg");
8 } // 析构自动释放
9
10 // 录像 10 秒
11 orix::OrixCamera cam;
12 cam.open();
13 cam.start_recording("output.mp4", 10.0);
14 std::this_thread::sleep_for(std::chrono::seconds(11));
15 cam.close();

```

9.10 数据类型参考

9.10.1 Vec3

三维向量，用于角速度（rad/s）、速度（m/s）等。

```

struct Vec3 {
    double x = 0.0;
    double y = 0.0;
    double z = 0.0;
};

```

9.10.2 Quat

四元数（Hamilton 约定：w, x, y, z）。

```

struct Quat {
    double w = 1.0;    // 实部
    double x = 0.0;    // 虚部 i

```

```
double y = 0.0;    // 虚部  $j$ 
double z = 0.0;    // 虚部  $k$ 
};
```

9.10.3 ImuData

IMU 惯性测量数据帧。

```
struct ImuData {
    Vec3    gyroscope;        // 角速度 ( $rad/s$ ), body frame
    Quat    quaternion;      // 姿态四元数 (Hamilton, world -> body)
    double  timestamp_ms = 0.0; // 相对会话启动的时间戳 ( $ms$ )
};
```

9.10.4 JointData

单个关节的实时数据。

```
struct JointData {
    int    id        = 0;    // 关节编号 (0-15)
    double position = 0.0;    // 角位置 ( $rad$ )
    double velocity = 0.0;    // 角速度 ( $rad/s$ )
    double torque   = 0.0;    // 输出力矩 ( $N\cdot m$ )
    int    status    = 0;    // 电机状态码 (0 = 正常)
};
```

9.10.5 MotorInfo

电机完整健康状态。

```
struct MotorInfo {
    int    id        = 0;    // 关节编号 (0-15)
    bool    online    = false; // 是否在线 (CAN 通信正常)
    double  temperature = 0.0; // 内部温度 ( $^{\circ}C$ )
    double  voltage     = 0.0; // 母线电压 ( $V$ )
    double  position    = 0.0; // 编码器位置 ( $rad$ )
    double  velocity    = 0.0; // 当前转速 ( $rad/s$ )
    double  torque      = 0.0; // 输出力矩 ( $N\cdot m$ )
    std::vector<int> errors;    // 故障码列表 (空 = 无故障)
};
```

9.10.6 ProfileInfo

运动策略信息。

```
struct ProfileInfo {
    std::string          current;          // 当前激活的策略名称
    std::vector<std::string> available;    // 所有可用策略名称
    std::vector<std::string> descriptions; // 各策略说明（与 available 对应）
};
```

9.10.7 GestureInfo

预设动作信息。

```
struct GestureInfo {
    std::string name;          // 动作名称（用于 play_gesture()）
    std::string description;   // 动作描述
    int         duration_ms;   // 预计执行时长（毫秒）
};
```

9.10.8 RobotState

FSM 状态枚举。使用 `orix::to_string(state)` 转为字符串。

```
enum class RobotState {
    Zero          = 0,    // "zero"          — 初始化/未使能
    Grounded      = 1,    // "grounded"        — 坐姿（安全状态）
    Standing      = 2,    // "standing"        — 站立
    Walking       = 3,    // "walking"         — 行走中
    Transitioning = 4,    // "transitioning"   — 过渡中
    Unknown       = -1,   // "unknown"
};

std::string to_string(RobotState s); // 返回上方字符串
```

```
1 auto state = dog.get_state();
2 if (state == orix::RobotState::Standing) {
3     dog.walk(0.3);
4 }
5 std::cout << orix::to_string(state) << "\n"; // "standing"
```

9.11 异常类型

所有 SDK 异常继承自 `orix::OrixError`，后者继承自 `std::runtime_error`。

```
class OrixError      : public std::runtime_error { ... };
class ConnectionError : public OrixError { ... }; // gRPC UNAVAILABLE
class InvalidStateError: public OrixError { ... }; // gRPC
                FAILED_PRECONDITION
class TimeoutError   : public OrixError { ... }; // gRPC
                DEADLINE_EXCEEDED
```

异常类	gRPC 状态码	含义
ConnectionError	UNAVAILABLE	无法连接机器人（IP 错误、服务未启动）
InvalidStateError	FAILED_PRECONDITION	机器人状态不满足前提（如 Zero 状态调 walk）
TimeoutError	DEADLINE_EXCEEDED	RPC 超时（网络延迟、服务无响应）
OrixError	其他	通用 SDK 错误

9.11.1 连接超时

```
1 try {
2     orix::OrixClient dog("192.168.66.190");
3     dog.stand_up();
4     dog.walk(0.5);
5 } catch (const orix::ConnectionError& e) {
6     std::cerr << "连接错误: " << e.what() << "\n";
7 } catch (const orix::InvalidStateError& e) {
8     std::cerr << "状态错误: " << e.what() << "\n";
9 } catch (const orix::TimeoutError& e) {
10    std::cerr << "超时: " << e.what() << "\n";
11 } catch (const orix::OrixError& e) {
12    std::cerr << "SDK 错误: " << e.what() << "\n";
13 }
```

9.11.2 常见错误码

gRPC 状态码	含义	常见原因
UNAVAILABLE	服务不可达	机器人未开机、IP 错误、服务未启动
DEADLINE_EXCEEDED	请求超时	网络延迟、服务端无响应
FAILED_PRECONDITION	前置条件不满足	状态不对（如 Zero 状态下调 walk）

gRPC 状态码	含义	常见原因
PERMISSION_DENIED	权限被拒	Arbiter 控制权被遥控器占用
INTERNAL	服务端内部错误	硬件异常或程序错误

9.11.3 Arbiter 控制权冲突

```

1 // 遥控器在线时 walk() 可能被忽略 (PERMISSION_DENIED)
2 // 以下模式可安全重试
3 auto try_walk = [&](double vx, int retries = 3) {
4     for (int i = 0; i < retries; ++i) {
5         try {
6             dog.walk(vx);
7             return true;
8         } catch (const orix::OrixError& e) {
9             std::cerr << "控制权被遥控器占用，等待 ("
10                << (i+1) << "/" << retries << ")...\n";
11             std::this_thread::sleep_for(std::chrono::seconds(1));
12         }
13     }
14     std::cerr << "无法获得控制权，请确认遥控器已松开摇杆。\\n";
15     return false;
16 };

```

9.11.4 流式接口异常处理

```

1 try {
2     dog.listen_imu([](const orix::ImuData& d) -> bool {
3         // 处理数据...
4         return true;
5     });
6 } catch (const orix::ConnectionError& e) {
7     std::cerr << "连接断开，尝试重连...\n";
8 } catch (const orix::OrixError& e) {
9     std::cerr << "流异常: " << e.what() << "\\n";
10 }

```

附录：方法速查表

分类	方法	返回值	说明
连接	OrixClient(host, port, timeout)	–	构造并连接
	~OrixClient()	–	析构释放 (RAII)
	ping()	bool	连接检测, 不抛异常
电机	enable()	void	使能全部电机
	disable()	void	禁用全部电机
运动	walk(vx, vy, vyaw)	void	行走指令, [-1,1], 自动 clamp
	stand_up()	void	坐姿 → 站立
	sit_down()	void	站立 → 坐姿
速度	set_high_speed(enabled)	void	高速模式 (最大 2.5 m/s)
	get_speed_mode()	string	"normal" 或 "high_speed"
状态	get_state()	RobotState	FSM 状态枚举
	get_voltage()	vector<double>	16 路母线电压 (V)
	get_motor_status()	vector<MotorInfo>	16 个电机健康状态
策略	get_profile()	ProfileInfo	当前策略及可用列表
	switch_profile(name)	ProfileInfo	切换策略 (须 Grounded)
动作	list_gestures()	vector<GestureInfo>	可用预设动作列表
	play_gesture(name, timeout)	void	播放预设动作 (须 Standing)
诊断	clear_motor_fault(ids)	void	清除故障码; 空 = 全部
	set_zero()	void	标定零位 (须 Grounded)
数据流	listen_imu(cb)	void (阻塞)	IMU 实时流, ~50 Hz
	listen_joint(cb)	void (阻塞)	关节实时流
	listen_state(cb)	void (阻塞)	FSM 状态变更事件

附录: OrixCamera 速查表

OrixCamera 是可选的摄像头辅助模块 (依赖 OpenCV), 提供拍照、录像和 MJPEG 流功能。

方法	返回值	说明
OrixCamera(source, width, height, fps)	–	本地 USB 摄像头
OrixCamera(url)	–	RTSP/MJPEG 远程流
open()	bool	打开摄像头
close()	void	释放摄像头设备
read()	cv::Mat	读取单帧图像
save_photo(path)	bool	拍照并保存
start_recording(path, duration, codec)	void	开始录像
stop_recording()	void	停止录像
stream_mjpeg(port, host)	void	启动 MJPEG HTTP 流服务器
stop_stream()	void	停止流服务器

```
1 #include "orix/orix_camera.h"
2
3 // 拍一张照片
4 orix::OrixCamera cam(0);
5 cam.open();
6 cam.save_photo("photo.jpg");
7 cam.close();
8
9 // MJPEG 流 (浏览器访问 http://localhost:8080/stream)
10 orix::OrixCamera cam(0);
11 cam.open();
12 cam.stream_mjpeg(8080);
13 std::this_thread::sleep_for(std::chrono::minutes(1));
14 cam.stop_stream();
15 cam.close();
```

Chapter 10

安全指南

本章导读

本章不是附录，而是实际调机和教学演示前必须执行的操作约束。很多“偶发故障”本质上都来自跳过检查、错误顺序或在不稳定环境中直接上动作。

阅读建议：任何涉及站起、行走、调零、清故障和切换策略的操作，建议先完成本章中的检查清单。

10.1 操作前检查清单

每次启动机器人前，必须逐项确认以下事项：

序号	检查项	确认方法	通过标准
1	电池电量充足	<code>dog.get_voltage()</code>	所有电机电压 $\geq 24V$
2	操作区域净空	目视检查	周围 1.5 米内无人/易碎物品
3	机器人放置在地面	目视检查	四脚均接触平坦地面
4	急停装置就绪	确认遥控器在手	遥控器已开机
5	CAN 总线连接牢固	目视检查	CAN 线缆无松动
6	控制服务已启动	SSH 检查	brainstem active
7	电机状态正常	<code>get_motor_status()</code>	16 电机在线无故障
8	关节位置合理	<code>get_motor_status()</code>	绝对值 $< 1.0 \text{ rad}$
9	遥控器优先级确认	操控遥控器摇杆	能正常控制
10	运行环境温度适宜	环境检查	$0-40^{\circ}\text{C}$

启动前检查代码：

```
1 #include "orix/orix.h"
2 #include <algorithm>
3 #include <cmath>
4 #include <iostream>
5
6 int main() {
7     orix::OrixClient dog("192.168.66.190");
8
9     // 1. 检查电压
```



```

10     auto voltages = dog.get_voltage();
11     double min_v = *std::min_element(voltages.begin(),
12                                     voltages.end());
13     std::cout << "电压: " << min_v << "V -- "
14               << (min_v >= 24.0 ? "通过" : "不通过!") << "\n";
15
16     // 2. 检查电机状态
17     auto motors = dog.get_motor_status();
18     int offline = 0, faulted = 0;
19     for (const auto& m : motors) {
20         if (!m.online) ++offline;
21         if (!m.errors.empty()) ++faulted;
22     }
23     std::cout << "电机在线: " << (16 - offline) << "/16 -- "
24               << (offline == 0 ? "通过" : "不通过!") << "\n";
25     std::cout << "电机故障: " << faulted << "/16 -- "
26               << (faulted == 0 ? "通过" : "需清除") << "\n";
27
28     // 3. 检查关节位置
29     bool pos_ok = true;
30     for (const auto& m : motors) {
31         if (std::abs(m.position) > 1.0) {
32             std::cout << "关节 " << m.id << " 位置异常: "
33                     << m.position << " rad\n";
34             pos_ok = false;
35         }
36     }
37     std::cout << "关节位置: "
38               << (pos_ok ? "通过" : "异常!") << "\n";
39
40     // 4. 检查状态
41     auto state = dog.get_state();
42     std::cout << "当前状态: " << orix::to_string(state)
43               << "\n";
44
45     return 0;
46 }

```

10.2 低压保护

当任一电机报告的母线电压低于 **18V** 时，系统自动触发低压保护：

1. 检测到任一电机母线电压低于 18V

2. 自动发送 SitDown 指令
3. 等待机器人进入 Grounded 状态
4. 禁用全部电机

电压范围	状态	建议
$\geq 24V$	正常	正常使用
$20V - 24V$	偏低	关注电量
$18V - 20V$	低电量	尽快结束任务
$< 18V$	危险	自动触发保护

10.3 关节限位保护

10.3.1 保护机制

每个关节的位置都受到绝对限位保护。默认限位为 3.14 rad（即 π ，约 180 度）。任一关节位置绝对值超过此阈值将立即触发故障（Fault）。

配置	环境变量	默认值
关节限位	HAN_DOG_JOINT_LIMIT_RAD	3.14 rad

10.3.2 触发后果

关节超限后：

1. 对应电机立即进入故障状态
2. 机器人可能失去该腿的控制，导致摔倒
3. 必须手动清除故障后才能恢复

10.3.3 预防措施

- 确保零位标定正确（`set_zero()` 在标准坐姿下执行）
- 不要在关节卡住或受阻的情况下强行运动
- 如果需要更严格的保护，可设置更小的限位值：

```
1 # Linux/macOS (在机器狗上设置)
2 export HAN_DOG_JOINT_LIMIT_RAD=2.5 # 约 143 度
```

10.4 电机故障处理

10.4.1 检测故障

```
1 auto motors = dog.get_motor_status();
2 for (const auto& m : motors) {
3     if (!m.errors.empty())
4         std::cout << "Joint " << m.id << " 故障码: ";
5     for (int e : m.errors) std::cout << e << " ";
6     if (!m.online)
7         std::cout << "Joint " << m.id << " 离线 (CAN 通信中断)";
8 }
```

10.4.2 清除故障

```
1 // 方法 1: 清除所有电机故障
2 dog.clear_motor_fault();
3
4 // 方法 2: 只清除特定关节
5 dog.clear_motor_fault({0, 1, 2}); // 只清除前右腿
```

10.4.3 故障处理流程

发现电机故障后，推荐按以下步骤处理：

1. 判断机器人是否正在运动中。
2. 如果在运动中：先调用 `dog.sit_down()` 安全坐下，等待进入 `Grounded` 状态，再调用 `dog.clear_motor_fault()` 清除故障，最后重新检查电机状态。
3. 如果未在运动中：直接调用 `dog.clear_motor_fault()` 清除故障，然后重新检查。
4. 如果故障清除成功，可恢复正常运动；如果故障持续存在，需检查硬件（CAN 线缆、电源、电机本体）。

10.4.4 反复出现的故障

如果 `clear_motor_fault()` 后故障立即重新出现，说明存在持续性硬件问题：

表现	可能原因	处理
单个电机反复故障	CAN 线缆松动	重新插拔对应 CAN 接头
多个电机同时故障	母线问题	检查电池和配电板
故障码伴随高温	电机过热	停止运动，等待冷却至 50°C 以下
关节位置跳变后故障	编码器异常	重新标零 <code>set_zero()</code>

10.5 急停程序

按紧急程度从低到高：

10.5.1 软件急停（SDK）

```
1 // 优雅急停
2 dog.walk();           // 停步
3 dog.sit_down();       // 坐下
4 // 等待 Grounded ...
5 dog.disable();        // 禁用电机
```

10.5.2 遥控器急停

立即操控遥控器摇杆——遥控器自动获得最高控制权，接管机器人。

注意：遥控器始终具有最高优先级，可在任何时刻覆盖 SDK 指令。

10.5.3 硬件断电

最紧急时直接关闭电池开关或拔出电池。注意断电跌落可能造成机械损伤。

10.5.4 急停优先级

急停方式按优先级从高到低排列：

优先级	方式	说明
1	硬件断电	物理级，不可覆盖
2	遥控器控制	Arbiter 保证最高软件优先级
3	SDK disable()	gRPC 调用
4	低压自动保护	系统自动触发

10.6 标准关机序列

```
1 #include "orix/orix.h"
2 #include <chrono>
3 #include <thread>
4
5 void safe_shutdown(orix::OrixClient& dog) {
6     try {
7         dog.walk();
```

```

8      dog.sit_down();
9      // 轮询等待 Grounded
10     for (int i = 0; i < 50; ++i) {
11         if (dog.get_state() == orix::RobotState::Grounded)
12             break;
13         std::this_thread::sleep_for(
14             std::chrono::milliseconds(200));
15     }
16     dog.disable();
17 } catch (...) {
18     try { dog.disable(); } catch (...) {}
19 }
20 }
```

10.7 Arbiter 控制仲裁安全

10.7.1 仲裁规则

ORIX 的 ControlArbiter（控制仲裁器）实施以下安全规则：

规则	说明
遥控器优先	遥控器的优先级永远高于 SDK 指令，3 秒无操作自动释放
超时释放	遥控器停止操作 3 秒后，gRPC 自动获得控制权
无抢占	gRPC 不能强行抢占遥控器的控制权
超时可配	超时时间通过 HAN_DOG_ARBITER_TIMEOUT 配置，默认 3 秒

10.7.2 对 SDK 开发者的影响

当 SDK 发送 walk() 等运动指令时，Arbiter 首先检查当前控制权归属。如果 gRPC 拥有控制权，指令正常执行；如果遥控器正在占用控制权，指令将被忽略或拒绝。

- 当遥控器在线并活跃时，SDK 的运动指令将被忽略
- 状态查询（get_state()、get_voltage() 等）不受 Arbiter 影响，始终可用
- 遥控器释放后约 3 秒，SDK 自动恢复控制权，无需额外操作

10.7.3 安全启示

- 现场测试时必须携带遥控器，作为最终的安全手段
- 不要依赖 SDK 作为唯一的急停方式
- 如果 SDK 指令无响应，先检查遥控器是否在线

10.8 禁止操作

以下操作严格禁止：

序号	禁止操作	原因
1	在桌面/高台上运行	摔落损坏
2	电机使能时翻转机器人	电机对抗损坏齿轮箱
3	电机使能时插拔 CAN 线	CAN 总线短路
4	长时间不监控温度	电机过热损坏绕组
5	关节卡死时反复运动	烧毁电机
6	多客户端同时发运动指令	指令冲突抖动

注意：ORIX 安全操作五条铁律：

1. 永远带遥控器 — 遥控器是最终安全手段
2. 先坐再断电 — `disable()` 前必须 `sit_down()`
3. 关注电压 — 低于 24V 准备收工
4. 监控温度 — 超过 70°C 立即休息
5. 地面运行 — 永远在地面操作

10.9 故障诊断树

遇到机器人异常时，按以下步骤逐级排查：

1. 机器人是否有响应？

- 无响应：检查电源（电池是否有电、开关是否打开）、检查网络连通性（ping 测试）、检查 CAN 线缆连接、SSH 是否可连通、brainstem 服务是否运行。
- 有响应：进入下一步。

2. 机器人是否在运动中？

- 在运动中：检查运动是否正常。如运动异常，参考下方运动异常诊断。
- 未运动：调用 `dog.get_state()` 查询状态。如可查询，再用 `dog.get_motor_status()` 检查电机状态和故障码；如不可查询，排查网络问题。

3. 有故障码：调用 `dog.clear_motor_fault()` 清除后重新检查。

4. 无故障码：检查状态机当前状态是否正确。

运动异常诊断：

1. 机器人摔倒：首先用 `get_voltage()` 检查电压。低于 18V 说明是低压保护触发；电压正常则检查关节限位是否超限。

2. 运动不流畅：用 `get_motor_status()` 检查电机温度。温度超过 70°C 说明过热，需暂停运动等待冷却。
3. 指令无效：检查 Arbiter 状态，确认遥控器是否在线占用控制权。

附录：安全相关环境变量

环境变量	默认值	说明
HAN_DOG_JOINT_LIMIT_RAD	3.14	关节绝对限位 (rad)
HAN_DOG_ARBITER_TIMEOUT	3	遥控器释放后 gRPC 获取控制权等待时间 (s)
HAN_DOG_SHUTDOWN_TIMEOUT	8	关机等待超时 (s)
HAN_DOG_STARTUP_TIMEOUT	10	FSM 启动等待 Grounded 超时 (s)
HAN_DOG_SENSOR_LOW_THRESHOLD	3	传感器低读数计数阈值

Chapter 11

常见问题

本章导读

本章按现场排障顺序组织，涵盖连接、编译、控制权、电压、状态机和 C++ 特有问题。
阅读建议：排障时请先确认现象，再执行对应的检查代码，不要一开始就同时修改多个变量。

11.1 连接问题

11.1.1 Q: 连接超时 (TimeoutError)

排查步骤：

```
1 # 1. 确认网络可达
2 ping 192.168.66.190
3
4 # 2. 确认端口开放
5 nc -zv 192.168.66.190 13145
6
7 # 3. 确认服务运行
8 ssh sunrise@192.168.66.190 "systemctl status brainstem"
```

原因	解决
机器人未开机	打开电池开关，等 30 秒
不在同一网段	确认局域网连通
brainstem 未启动	<code>systemctl start brainstem</code>
防火墙拦截	放行 13145 端口
超时太短	构造时增大 timeout

11.1.2 Q: 连接被拒绝 (ConnectionError)

```
1 try {
2     orix::OrixClient dog("192.168.66.190");
```



```
3     dog.get_state();
4 } catch (const orix::ConnectionError& e) {
5     std::cerr << "连接失败: " << e.what() << "\n";
6     // 检查 brainstem 服务、IP 地址和端口
7 }
```

11.1.3 Q: IP 地址不确定

如果不确定机器人 IP:

```
1 # 方法 1: 通过路由器管理页面查看 DHCP 租约
2
3 # 方法 2: 在已知网段扫描 gRPC 端口 (Linux, 需安装 nmap)
4 nmap -p 13145 192.168.66.0/24
5
6 # 方法 3: SSH 登录后查看 IP
7 ip addr show
```

默认 IP 为 192.168.66.190, 如果网络配置未更改, 直接使用即可。

11.2 编译问题

11.2.1 Q: CMake 找不到 gRPC

```
1 # 手动指定路径
2 cmake .. -DCMAKE_PREFIX_PATH=/path/to/grpc/install
3
4 # 或使用 vcpkg
5 cmake .. -DCMAKE_TOOLCHAIN_FILE=\
6     /path/to/vcpkg/scripts/buildsystems/vcpkg.cmake
```

11.2.2 Q: 链接错误: undefined reference to grpc

确保 `target_link_libraries` 中包含了所有必要的库:

```
1 target_link_libraries(my_app PRIVATE
2     orix::orix
3     gRPC::grpc++
4     protobuf::libprotobuf
5     pthread
6 )
```

11.2.3 Q: C++17 标准不满足

确认编译器版本和 CMake 配置:

```
1 g++ --version # 需要 GCC 9+
```

```
1 set(CMAKE_CXX_STANDARD 17)
2 set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

11.3 电机不动

11.3.1 Q: enable() 后电机没反应

```
1 // 1. 检查状态
2 auto state = dog.get_state();
3 std::cout << "状态: " << orix::to_string(state) << "\n";
4
5 // 2. 检查电机在线
6 auto motors = dog.get_motor_status();
7 for (const auto& m : motors) {
8     if (!m.online)
9         std::cout << "Joint " << m.id << " 离线\n";
10    if (!m.errors.empty())
11        std::cout << "Joint " << m.id << " 有故障\n";
12 }
13
14 // 3. 检查电压
15 auto voltages = dog.get_voltage();
16 double min_v = *std::min_element(voltages.begin(),
17                                   voltages.end());
18 std::cout << "最低电压: " << min_v << "V\n";
```

11.3.2 Q: 只有部分电机动, 其他不动

```
1 auto motors = dog.get_motor_status();
2 for (const auto& m : motors) {
3     const char* status = (m.online && m.errors.empty()) ? "OK" : "FAIL";
4     std::cout << "Joint " << m.id << ": " << status
5               << " online=" << m.online << "\n";
6 }
```

常见原因：

- 对应腿的 CAN 线缆断开——检查该腿的 CAN 总线物理连接
- 单个电机驱动板故障——更换后重试
- 电机过热保护——查看温度值，等待冷却

11.3.3 Q: enable() 后状态没变化

enable() 使能电机，但不会改变 FSM 状态。使能后需要手动切换状态：

```
1 dog.enable();
2 // 此时状态仍为 Grounded，电机通电但保持坐姿
3 dog.stand_up();
4 // 现在状态变为 Standing
```

11.4 行走指令无效

11.4.1 Q: walk() 不生效

必须按状态机顺序操作：

```
1 auto state = dog.get_state();
2 if (state == orix::RobotState::Grounded) {
3     std::cout << "需要先站起来\n";
4     dog.stand_up();
5     std::this_thread::sleep_for(std::chrono::seconds(4));
6 }
7 if (dog.get_state() == orix::RobotState::Standing) {
8     dog.walk(0.3); // 现在应该生效
9 }
```

11.4.2 Q: 遥控器在线时 SDK 指令被忽略

这是正常行为。ControlArbiter 保证遥控器最高优先级。关闭遥控器或松开摇杆 3 秒后 SDK 自动恢复控制权。

11.5 机器人摔倒

11.5.1 Q: 行走中突然坐下

最可能原因：低压保护触发。

```
1 auto voltages = dog.get_voltage();
```

```

2 double min_v = *std::min_element(voltages.begin(), voltages.end());
3 std::cout << "当前电压: " << min_v << "V\n";
4 // 如果 < 18V, 说明是低压保护

```

其他可能原因:

原因	诊断方法
低电压 (< 18V)	get_voltage() 返回值低于 18
电机过热	get_motor_status() 中 temperature > 80°C
关节超限	get_motor_status() 中 position 绝对值过大
CAN 通信中断	get_motor_status() 中部分电机 online=false

11.5.2 Q: 站起来后立即摔倒

排查:

1. 地面是否太滑——在有摩擦力的地面（橡胶垫、地毯）上测试
2. 零位是否正确——如果关节零位偏移，站立姿态会畸形
3. 策略是否匹配——确认加载的运动策略与当前机器人硬件配置一致

11.6 策略切换失败

11.6.1 Q: switch_profile() 报错

前置条件：策略切换只能在 Grounded 状态下执行。

```

1 auto state = dog.get_state();
2 if (state != orix::RobotState::Grounded) {
3     std::cout << "当前状态: " << orix::to_string(state)
4         << ", 需要先坐下.\n";
5     dog.walk();
6     dog.sit_down();
7     // 等待进入 Grounded
8     for (int i = 0; i < 50; ++i) {
9         if (dog.get_state() == orix::RobotState::Grounded) break;
10        std::this_thread::sleep_for(std::chrono::milliseconds(200));
11    }
12 }
13 // 现在可以切换
14 auto profile = dog.switch_profile("mini");
15 std::cout << "已切换到: " << profile.current << "\n";

```

11.6.2 Q: 策略名称不存在

```

1 // 先查看可用策略
2 auto profile = dog.get_profile();
3 std::cout << "可用策略: ";
4 for (const auto& p : profile.available) std::cout << p << " ";
5 std::cout << "\n";
6
7 // 使用列表中的名称
8 dog.switch_profile(profile.available[0]);

```

11.6.3 Q: 切换策略后行为异常

每个策略针对特定硬件配置和运动模式进行训练。切换策略后如果出现异常运动:

1. 确认策略与当前硬件配置匹配
2. 立即坐下: `dog.sit_down()`
3. 切回之前正常工作的策略

11.7 IMU 数据异常

11.7.1 Q: 四元数值看起来不对

ORIX SDK 使用 Hamilton 约定, 四元数顺序为 (w, x, y, z):

```

1 dog.listen_imu([](const orix::ImuData& d) -> bool {
2     // SDK 返回: (w, x, y, z)
3     std::cout << "w=" << d.quaternion.w
4                 << " x=" << d.quaternion.x
5                 << " y=" << d.quaternion.y
6                 << " z=" << d.quaternion.z << "\n";
7     return true;
8 });

```

如果你的应用使用 ROS 约定 (x, y, z, w), 需要手动转换:

```

1 // SDK (Hamilton) -> ROS
2 // ros_quat = {d.quaternion.x, d.quaternion.y,
3 //              d.quaternion.z, d.quaternion.w}

```

11.7.2 Q: 机器人静止但角速度不为零

IMU 的陀螺仪存在零偏 (bias)，静止时读数不会严格为零。典型零偏范围： $|\text{gyro.x}| < 0.02 \text{ rad/s}$ 属于正常； $|\text{gyro.x}| > 0.05 \text{ rad/s}$ 可能需要重新标定。

如需高精度角速度，可在启动时采集 100 帧静止数据取均值作为零偏，后续数据减去此偏移。

11.7.3 Q: listen_imu() 频率不是 50Hz

IMU 标称频率约 50Hz，实际可能在 48–52Hz 波动，这是正常的。如果频率显著偏低 ($< 30\text{Hz}$)，检查：

- 串口连接是否正常
- brainstem 服务是否负载过高
- 网络延迟 (gRPC 传输开销)

11.8 流式接口问题

11.8.1 Q: listen_imu() 阻塞了主线程

所有 listen_* 方法都是阻塞的。使用独立线程：

```
1 std::atomic<bool> running{true};
2
3 std::thread imu_thread([&] {
4     dog.listen_imu([&](const orix::ImuData& d) -> bool {
5         // 处理数据...
6         return running.load();
7     });
8 });
9
10 // 主线程继续执行其他操作...
11
12 running = false;
13 imu_thread.join();
```

11.8.2 Q: 流中断后如何重连

当网络断开或服务重启时，流式回调会结束。重新创建 OrixClient 并重新订阅：

```
1 while (true) {
2     try {
3         orix::OrixClient dog("192.168.66.190");
4         dog.listen_imu([&](const orix::ImuData& d) -> bool {
```

```

5          // 处理数据...
6          return true;
7      });
8  } catch (const orix::ConnectionError& e) {
9      std::cerr << "连接断开, 5 秒后重试...\n";
10     std::this_thread::sleep_for(std::chrono::seconds(5));
11 }
12 }

```

11.9 性能与延迟

11.9.1 Q: C++ SDK 比 Python 快多少?

指标	Python SDK	C++ SDK
单次 RPC 延迟	1–5 ms	0.5–2 ms
流式数据处理	受 GIL 限制	真正多线程并行
启动时间	~500 ms	~50 ms
内存占用	~50 MB	~5 MB
适用场景	快速原型、教学	嵌入式部署、高频控制

11.9.2 Q: 50Hz 控制频率够用吗?

ORIX 的运动控制循环固定为 50Hz。SDK 发送的指令在下一个控制周期生效。不需要以超过 50Hz 的频率发送 `walk()` 指令。C++ 的低延迟特性在需要精确时序控制的场景中尤为重要。

11.10 环境变量参考

以下环境变量在机器人端（brainstem 服务）生效，SDK 客户端不直接使用。了解这些变量有助于理解系统行为和排障。

环境变量	类型	默认值	说明
HAN_DOG_PORT	int	13145	gRPC 服务监听端口
HAN_DOG_CONTROLLER	str	auto	遥控器设备（自动检测）
HAN_DOG_DEFAULT_PROFILE	str	（无）	默认加载的运动策略
HAN_DOG_PROFILE_DIR	str	profiles/	策略文件目录
HAN_DOG_ARBITER_TIMEOUT	int	3	遥控器释放后控制权切换 (s)
HAN_DOG_SENSOR_LOW_THRESHOLD	int	3	传感器低读数计数阈值
HAN_DOG_SHUTDOWN_TIMEOUT	int	8	关机等待超时 (s)
HAN_DOG_STARTUP_TIMEOUT	int	10	启动等待 Grounded 超时 (s)

环境变量	类型	默认值	说明
HAN_DOG_JOINT_LIMIT_RAD	float	3.14	关节绝对限位 (rad)
HAN_DOG_LOG_DIR	str	logs	日志文件目录
HAN_DOG_LOG	str	INFO	日志级别
MEDULLA_STANDALONE	str	(无)	1 = 启用独立模式

11.11 摄像头问题

11.11.1 Q: OrixCamera 编译失败

OrixCamera 需要 OpenCV >= 4.5。确认安装：

```
1 # vcpkg
2 ./vcpkg install opencv4
3
4 # 或使用系统包管理器
5 sudo apt install libopencv-dev
```

11.11.2 Q: 远程访问摄像头

开发机上可通过 RTSP URL 访问机器人摄像头：

```
1 // 远程模式
2 orix::OrixCamera cam("rtsp://192.168.66.190:8554/cam");
3 cam.open();
4 cv::Mat frame = cam.read();
```

11.12 网络排障

```
1 # 1. Ping 机器人
2 ping 192.168.66.190
3
4 # 2. 检查端口
5 nc -zv 192.168.66.190 13145
6
7 # 3. 检查服务
8 ssh sunrise@192.168.66.190 "systemctl status brainstem"
9
10 # 4. 远程开发 (SSH 隧道)
11 ssh -L 13145:192.168.66.190:13145 sunrise@192.168.66.190
```



```
12 # 然后连接 localhost
```

11.13 提交 Bug 报告

遇到无法自行解决的问题时，请按以下格式提交 Bug 报告。

11.13.1 必要信息

1. SDK 版本: orix C++ SDK v1.0.0
2. 编译器版本: `g++ --version` / `cl --version`
3. 操作系统: (例: Ubuntu 22.04 / Windows 11)
4. 机器人固件版本: (SSH 登录后查看 `brainstem` 版本)
5. 问题描述: (清晰描述预期行为和实际行为)
6. 复现步骤: (从连接开始, 逐步列出操作)
7. 错误信息: (完整的异常信息或错误输出)
8. 机器人日志: (SSH 登录, 提供 `logs/han_dog_YYYYMMDD.log` 中相关片段)

11.13.2 收集诊断信息

```
1 #include "orix/orix.h"
2 #include <iostream>
3
4 int main() {
5     std::cout << "=== ORIX C++ SDK 诊断信息 ===\n";
6     std::cout << "SDK 版本: 1.0.0\n";
7
8     try {
9         orix::OrixClient dog("192.168.66.190", 13145, 5.0);
10
11         auto state = dog.get_state();
12         std::cout << "连接:      成功\n";
13         std::cout << "机器人状态: " << orix::to_string(state) << "\n";
14
15         auto voltages = dog.get_voltage();
16         if (!voltages.empty()) {
17             double min_v = *std::min_element(
18                 voltages.begin(), voltages.end());
19             double max_v = *std::max_element(
20                 voltages.begin(), voltages.end());
21             std::cout << "电压范围: " << min_v << "V ~ "
22                 << max_v << "V\n";
```

```

23     }
24
25     auto motors = dog.get_motor_status();
26     int online = 0, faults = 0;
27     double max_temp = 0;
28     for (const auto& m : motors) {
29         if (m.online) ++online;
30         if (!m.errors.empty()) ++faults;
31         if (m.temperature > max_temp) max_temp = m.temperature;
32     }
33     std::cout << "电机在线:  " << online << "/16\n";
34     std::cout << "电机故障:  " << faults << "/16\n";
35     std::cout << "最高温度:  " << max_temp << "C\n";
36
37     auto profile = dog.get_profile();
38     std::cout << "当前策略:  " << profile.current << "\n";
39 } catch (const orix::ConnectionError& e) {
40     std::cerr << "连接:      失败 - " << e.what() << "\n";
41 } catch (const std::exception& e) {
42     std::cerr << "异常:      " << e.what() << "\n";
43 }
44
45 std::cout << "=====\n";
46 std::cout << "请将以上输出复制到 Bug 报告中。 \n";
47 return 0;
48 }

```

11.13.3 日志收集

```

1 # SSH 登录机器人
2 ssh sunrise@192.168.66.190
3
4 # 查看今天的日志
5 cat logs/han_dog_$(date +%Y%m%d).log
6
7 # 只看错误和警告
8 grep -E "WARNING|SEVERE" logs/han_dog_$(date +%Y%m%d).log
9
10 # 查看最近 100 行
11 tail -100 logs/han_dog_$(date +%Y%m%d).log
12
13 # 导出日志到本地（在开发机上执行）

```

```
14 scp sunrise@192.168.66.190:logs/han_dog_*.log ./han_dog_logs/
```

11.13.4 联系方式

- GitHub Issues: 在 [brainstem](#) 仓库提交 Issue
- 附上诊断程序输出和相关日志

11.14 SSH 访问与系统服务

大多数排障操作都需要 SSH 登录机器人查看日志或重启服务。出厂镜像默认已启用 SSH, 无需额外配置。

11.14.1 默认账户

项目	默认值
用户名	sunrise
密码	sunrise
IP 地址	192.168.66.190
SSH 端口	22

```
1 ssh sunrise@192.168.66.190
2 # 提示输入密码: sunrise
```

11.14.2 brainstem 服务管理

ORIX 的控制栈 (brainstem) 以 systemd 服务形式运行, 开机自启。以下命令在 SSH 登录后执行:

```
1 # 查看服务状态
2 systemctl status brainstem
3
4 # 查看最近 100 行日志
5 journalctl -u brainstem -n 100
6
7 # 实时跟踪日志
8 journalctl -u brainstem -f
9
10 # 重启服务 (通常在切换策略或修改配置后)
11 sudo systemctl restart brainstem
```

```
12
13 # 停止服务（调试时临时关闭控制栈）
14 sudo systemctl stop brainstem
15
16 # 启动服务
17 sudo systemctl start brainstem
```

11.14.3 无网络环境下的备选连接方式

如果 SSH 被校园网或企业防火墙拦截，或机器人未能获取 DHCP 地址，可以使用 RJ45 网线直连：

1. 用网线将开发机直接连接到机器人的 RJ45 口
2. 将开发机的以太网接口配置为静态 IP，例如 192.168.66.100/24
3. 使用默认地址连接：`ssh sunrise@192.168.66.190`

这种方式不依赖任何路由器或交换机，是调试现场最可靠的连接手段。

附录：快速排障速查表

症状	首先检查	可能原因	解决方案
连接超时	ping 机器人	网络不通	检查网线/Wi-Fi/IP
连接被拒	brainstem 服务	服务未启动	<code>systemctl start brainstem</code>
编译失败	CMake 配置	缺少 gRPC	安装 <code>vcpkg</code> 依赖
enable() 无效	<code>get_motor_status()</code>	电机离线	检查 CAN 线
walk() 无效	<code>get_state()</code>	不在 Standing	先 <code>stand_up()</code>
walk() 被忽略	遥控器状态	Arbiter 被占用	松开遥控器
突然坐下	<code>get_voltage()</code>	低压 (< 18V)	充电
流阻塞主线程	代码结构	未用独立线程	<code>std::thread</code>
策略切换失败	<code>get_state()</code>	不在 Grounded	先 <code>sit_down()</code>